

4 ödev - 3 Lab - 1 Proje - 1 Final - 2 Uze
 $\hookrightarrow$ %14 $\downarrow$ %6 $\hookrightarrow$ %40 $\hookrightarrow$ %40

İyi algoritma $\rightarrow$ Zaman ve hafıza, iyi kullanır, adapte edilebilir, Basittir, Doğrudur, Optimum, düşük bağımlılık

Zaman ölçme

```
begin = clock()
*/
/*
end = clock()
timeSpent = (double) (end - begin) / CLOCKS_PER_SEC
```

} Empirik analiz

Sistem Bağımlı etkiler $\rightarrow$ Yazılım, donanım, işletim sistemi, internet vs.

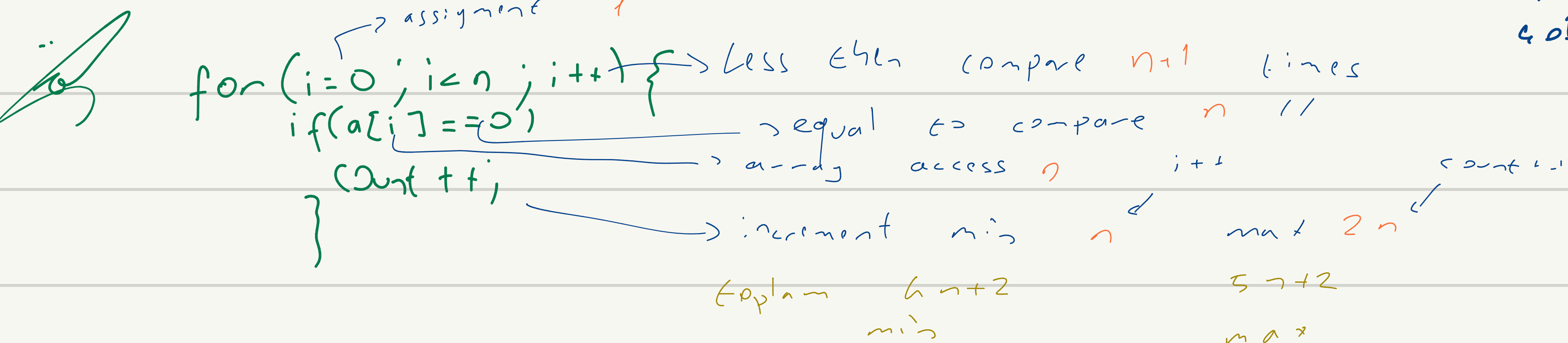
" Bağımsız " $\rightarrow$ Algoritma $\rightarrow$ Teorik analiz

switch vs if $\rightarrow$ az elemanda daha hizli.
 $\downarrow$
 got element olursa lookup table olusturuyor ve hizli oluyor
 $\hookrightarrow$ hashle

Analysis of Asymptotic Analysis Order of growth $n \rightarrow \infty$

Basic operation $\rightarrow$ En karmaşık işlem

düşük değerlerde
çok fazla



- ignore constant terms $\rightarrow 5n^2 + 3n \rightarrow$ not 10^{10} gibi bir sayı varsa onu da at!!!
 $\rightarrow$ bunu görmezden gelme!!!
- ignore coefficients $\rightarrow n^2 + n \rightarrow$ not $(n \times n)$
- pick the most significant term $\rightarrow n^2$

Asymptotic Notation $n \rightarrow \infty$

$O()$: Big- O
 $\downarrow$
 max

$\Theta()$: Big theta
 $\downarrow$
 average

$\Omega()$: Big Omega
 $\downarrow$
 min

1) Big- O notation

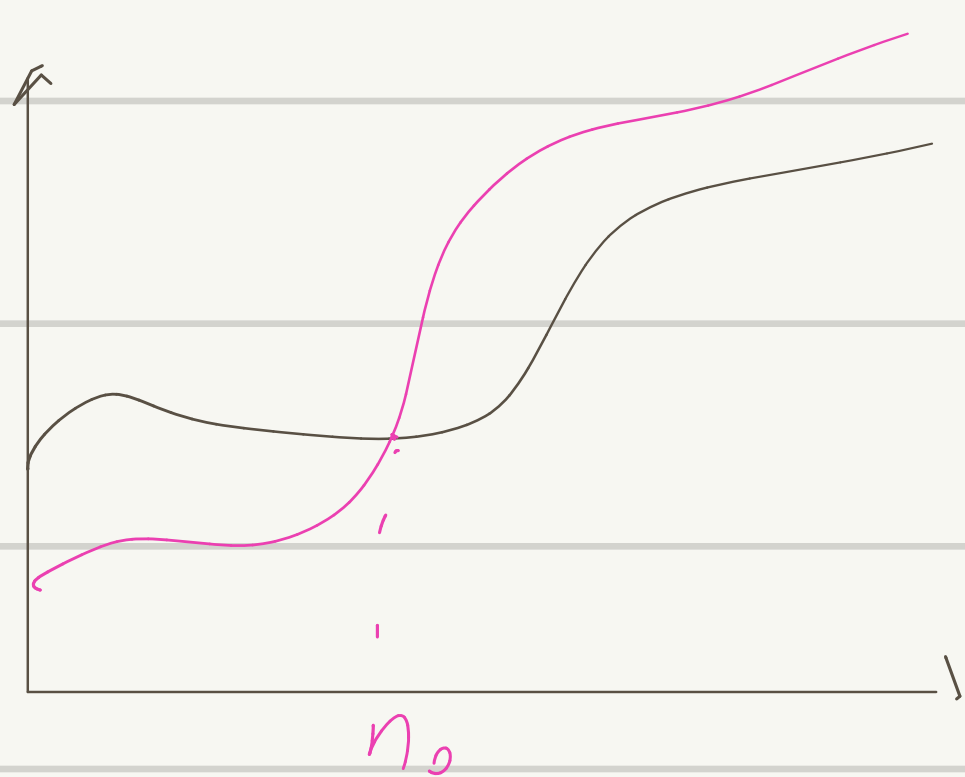
$$f(n) = 10n + 30 \rightarrow O(n)$$

$f(n) \in O(g(n))$ $f(n)$ böyle yazılabilir ama ne zaman

if $0 \leq f(n) \leq c \cdot g(n)$
 then

for all $c > 0, n \geq n_0 > 0$

$O(g(n))$: upper bound on the growth of a function $f(n)$



$$0 \leq 10n + 30 \leq c \cdot g(n) \rightarrow c \cdot n$$

$$30 \leq (c - 10) \cdot n$$

$$\frac{30}{c-10} \leq n$$

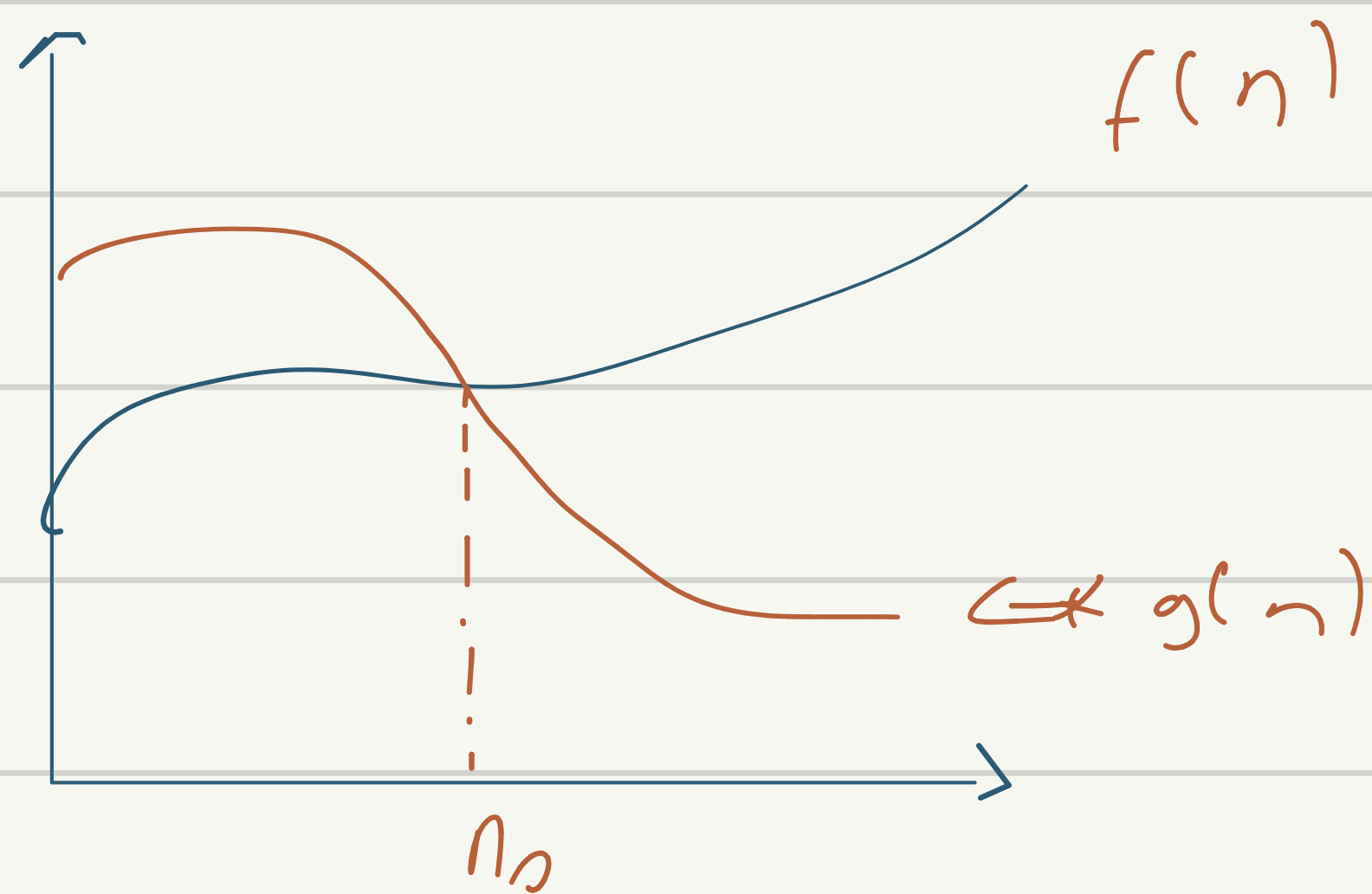
$$c = 11$$

$$30 \leq 30 \text{ sağlandı}$$

not n^2, n^3, n^4, n^5 her şey olur

② Big Omega notation lower bound

If $f(n) \geq c \cdot g(n) \geq 0$ for all $c > 0$ $n \geq n_0 > 0$
 then $f(n) \rightarrow \Omega(g(n))$



$$5n^2 \in \Omega(n)$$

$$5n^2 \geq c \cdot n \geq 0$$

$$n_0 = 1 \quad c = 3$$

$$5 \geq c \geq 0$$

$4n^2 \in \Omega(n^2)$

$$4n^2 \geq c \cdot n^2$$

$$c = 3$$

$$n_0 = 1$$

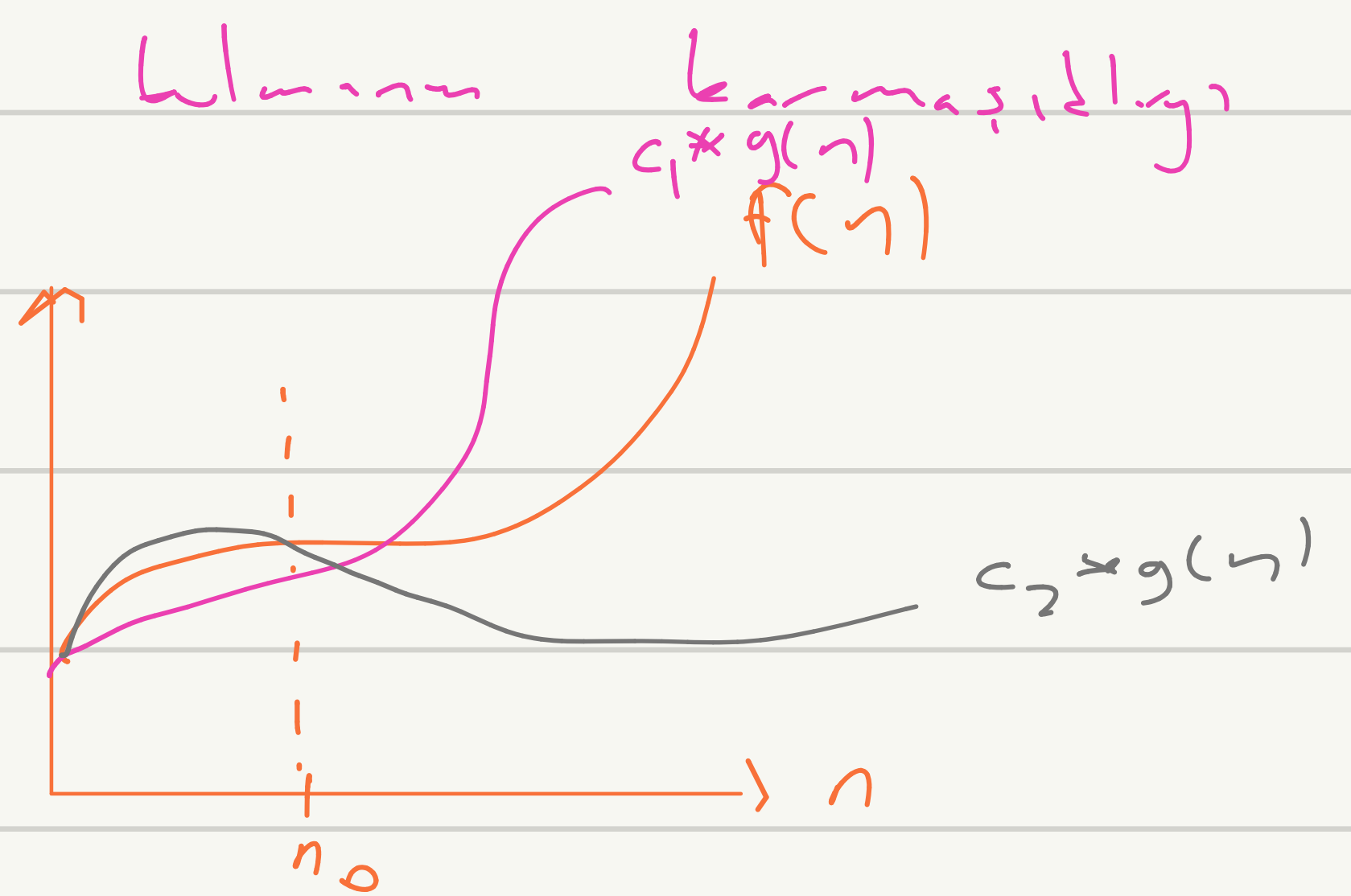
n_0 en düğün olmak
 zorunda değil

③ Big Theta tight bound

Bir dizinin en büyük elemanını bulmanın karmaşıklığı
 worst $\rightarrow N$ Best $\rightarrow N$

If $0 \leq c_2 \cdot g(n) \leq f(n) \leq c_1 \cdot g(n)$
 for all $c_2 \leq c_1$ $c_1 > 0$, $c_2 > 0$

then $f(n) \in \Theta(g(n))$



$f(n)$ is asymptotically equal to $g(n) \rightarrow f(n) \in \Theta(g(n))$

$$\left. \begin{array}{l} f(n) \in O(g(n)) \\ f(n) \in \Omega(g(n)) \end{array} \right\} \Theta(g(n))$$

$f(n) = 2n + 5 \rightarrow \Theta(n)$

$$n_0 = 1 \quad c_1 = 7 \quad 2n + 5 \leq c_1 \cdot g(n) \rightarrow O(n)$$

$$n_0 = 1 \quad c_1 = 1 \quad 2n + 5 \geq c_1 \cdot g(n) \rightarrow \Omega(n)$$

$$7 > 1$$

① Rekürsif olmayan algoritmaların analizi:

int findMax(int n, int *A) { ① Algoritmanın basit $\frac{n}{2}$ operasyonunu belirle.

int i, max;

② Ω, Θ, O 'sı n için belirle.

max = A[0];

for (i=1; i < n; i++) {

if (max < A[i])

max = A[i];

return max; }

basit operasyon

karşılaştırma

$$Sagısı = \sum_{i=1}^{n-1} 1 = n-1 \rightarrow n=2$$

$\downarrow$ best

$\downarrow$ worst

$\downarrow$ average

$O(n)$

$\Omega(n) = \Theta(n)$

$\Theta(n)$

↑
rastgele
elementlerin
ortalaması

② Sequential-Linear search

int linearSearch(int n, int *A, int v) {

int i = 0;

while (i < n && A[i] != v) {

i++;

if (i < n)

return i;

return -1; }

basic operation

$\Omega(n)$ t=1

$\Omega(n)$ worst = n

avg $\rightarrow$ p: $0 < p < 1$ element var i^{th} position $\frac{p}{n}$

$$C_{avg}(n) = \left[1 * \frac{p}{n} + 2 * \frac{p}{n} + \dots + n * \frac{p}{n} \right] + n * (1-p)$$

varsa $\rightarrow \frac{p * (n+1) * n}{2}$

yoksa

$$\frac{1 * n + 1}{2} + n * (1-1) \rightarrow \Theta(n)$$

$$p=0 \text{ için } \frac{0 * (n+1)}{2} + n * (1-0) \rightarrow n$$

$\checkmark$ $p=1$ ise yani element varsa

$\hookrightarrow C_{avg}(n) = n \rightarrow O(n) \rightarrow$ best & worst olduğu için
 $\Omega(1) \rightarrow \Theta$ yok

insertion Sort

9	3	7	1	4	5
3	9	7	1	4	5
3	7	9	1	4	5
1	3	7	9	4	5

⋮

1 3 4 5 7 9

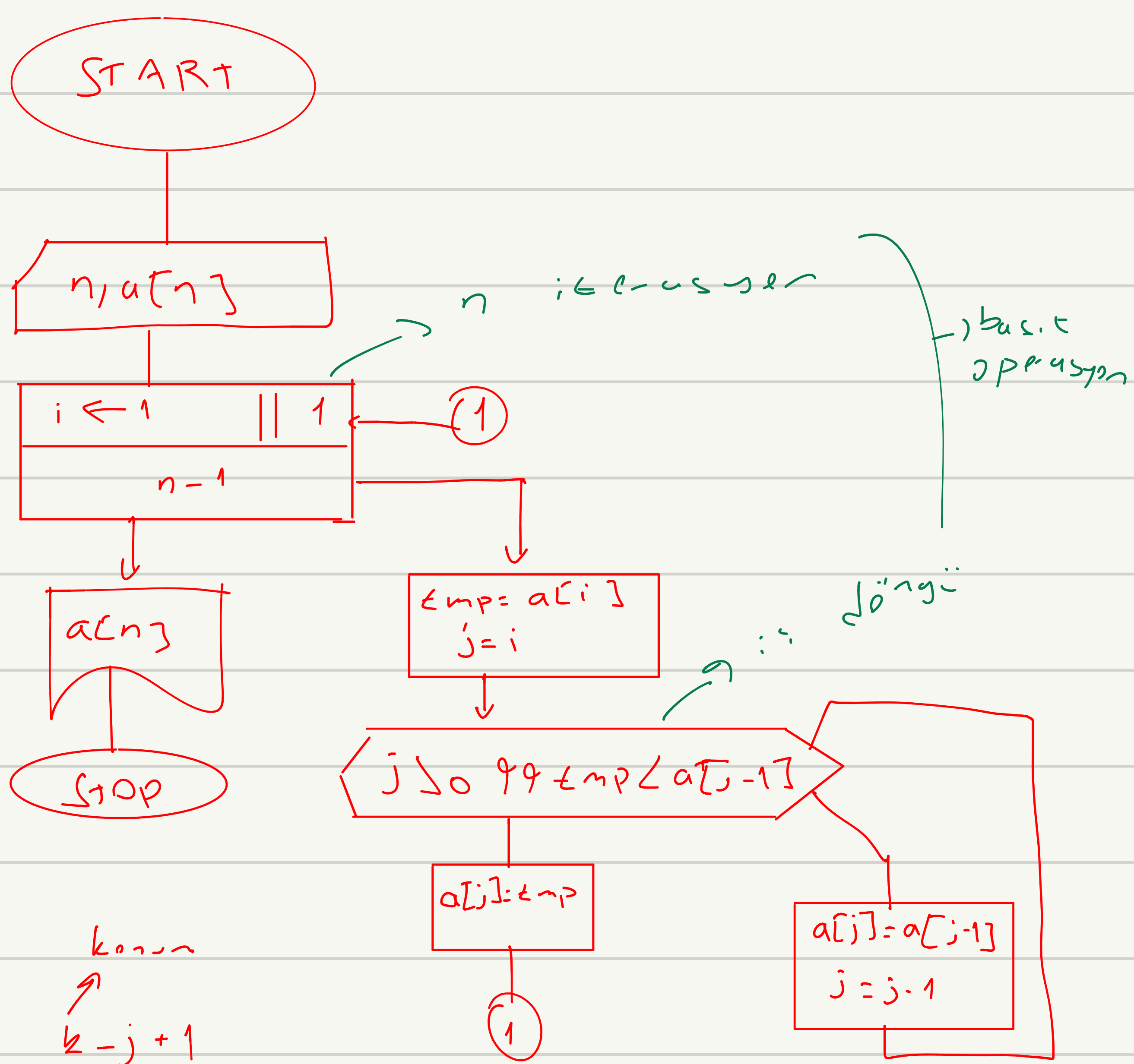
$C(n) \rightarrow$ sıralı dizi $\rightarrow n$
 $\rightarrow 1 \ 2 \ 3 \ 4$
 $i=1 \ 1$
 $i=2 \ 1$
 $i=3 \ 1$
 $i=n-1 \ 1$
 $\sum_{i=1}^{n-1} 1 \rightarrow n-1 \approx n$

$C_{\text{wurst}}(n) \rightarrow$ en kötü sıralı $\rightarrow n^2$
 karşılaştırma sayısı
 $i=1 \rightarrow 1$
 $i=2 \rightarrow 2$
 $i=n-1 \rightarrow n-1$
 $\sum_{i=1}^{n-1} i = \frac{(n-1)n}{2} \approx n^2$

$C_{\text{average}} \rightarrow$ karşılaştırma sayısı $k-j+1$

$$\frac{1}{k} \sum_{j=1}^k (k-j+1) \rightarrow \frac{1}{k} \left(k^2 - \frac{k(k+1)}{2} + k \right) \rightarrow k - \frac{(k+1)}{2} + 1 \rightarrow \frac{k+1}{2} \rightarrow \text{inner loop}$$

dış döngü: $\sum_{k=1}^{n-1} \frac{k+1}{2} \rightarrow \frac{n(n-1)}{2} + \frac{n-1}{2} \approx n^2$



Rekürsif için

$n! = 1 * 2 * 3 * \dots * n$
 $n! = n * (n-1)!$
 $f(n) = \begin{cases} 1 & n=0 \\ n * f(n-1) & n > 0 \end{cases}$

Rekürsif Bağlantısı
 çalışma karmaşıklığı $M(n)$
 $M(n) = M(n-1) + 1$
 $M(0) = 0 \rightarrow \sum_{i=1}^n 1 \rightarrow n$
 $\Omega(n) = \Theta(n) = O(n) = n$

1) Factorial

```

int factorial(int n) {
    if (n == 0)
        return 1;
    else
        return n * factorial(n-1);
}
    
```

- 1) Basic operation bul
- 2) Rekürsif bağlantısı yaz-söle

2 yol var 1. si: Backward Substitution $\rightarrow$ Geriye doğru yazma
 $M(n) = M(n-1) + 1$
 $M(n-1) = M(n-2) + 1$
 $\vdots$
 $M(0) = 0$
 $M(n) = M(n-3) + 3 \rightarrow M(n) = M(n-i) + i$
 $M(n) = M(n-n) + n$
 $M(n) = n$

Find the number of binary digits positive

$$\begin{array}{r}
 27 \mid 2 \\
 \hline
 1 \mid 13 \mid 2 \\
 \hline
 1 \mid 6 \mid 2 \\
 \hline
 0 \mid 3 \mid 2 \\
 \hline
 1 \mid 1
 \end{array}$$

$$\rightarrow \text{rec}(n) = \text{rec}(n/2) + 1$$

int binaryDigits(int n) {

if (n == 1)
return 1;

else
return 1 + binaryDigits(n/2); }

↳ basic op

$$\begin{aligned}
 A(n) &= A(n/2) + 1 & n \geq 1 & \rightarrow A(n/2) = A(n/4) + 1 & \rightarrow A(n) = A(n/8) + 3 \\
 A(1) &= 0 & A(n/4) &= A(n/8) + 2 & A(n) = A(n/2^i) + i \\
 \rightarrow A(n) &= A(2^i/2^i) + i & 2^i &= n & i = \log_2 n & A(n) = \log_2 n & \Theta(\log_2 n)
 \end{aligned}$$

forward substitutions

$$T(n) = \begin{cases} 1 & \text{if } n=1 \\ T(n-1)+1 & \text{otherwise} \end{cases}$$

n	$T(n)$
1	1
2	$T(1) + 1 \rightarrow 2$
3	3
4	4
$\vdots$	$\vdots$
n	n

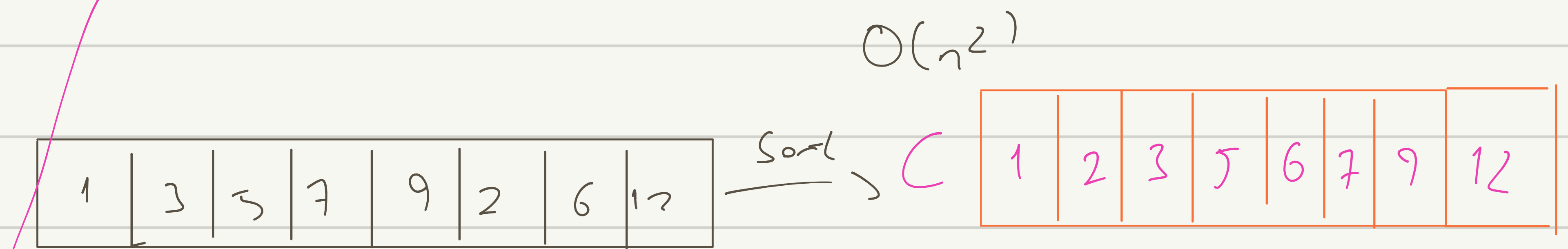
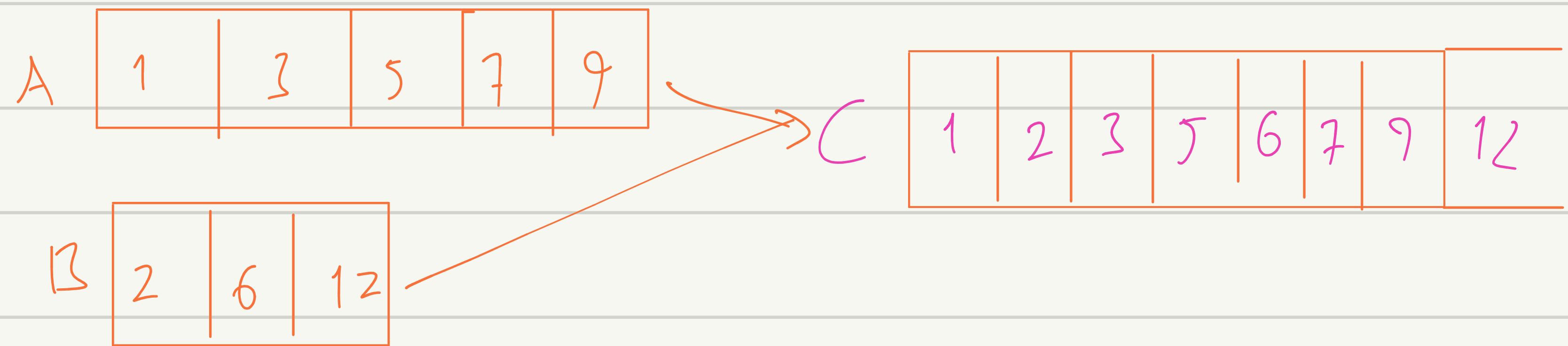
4. Hafta

Insertion Sort

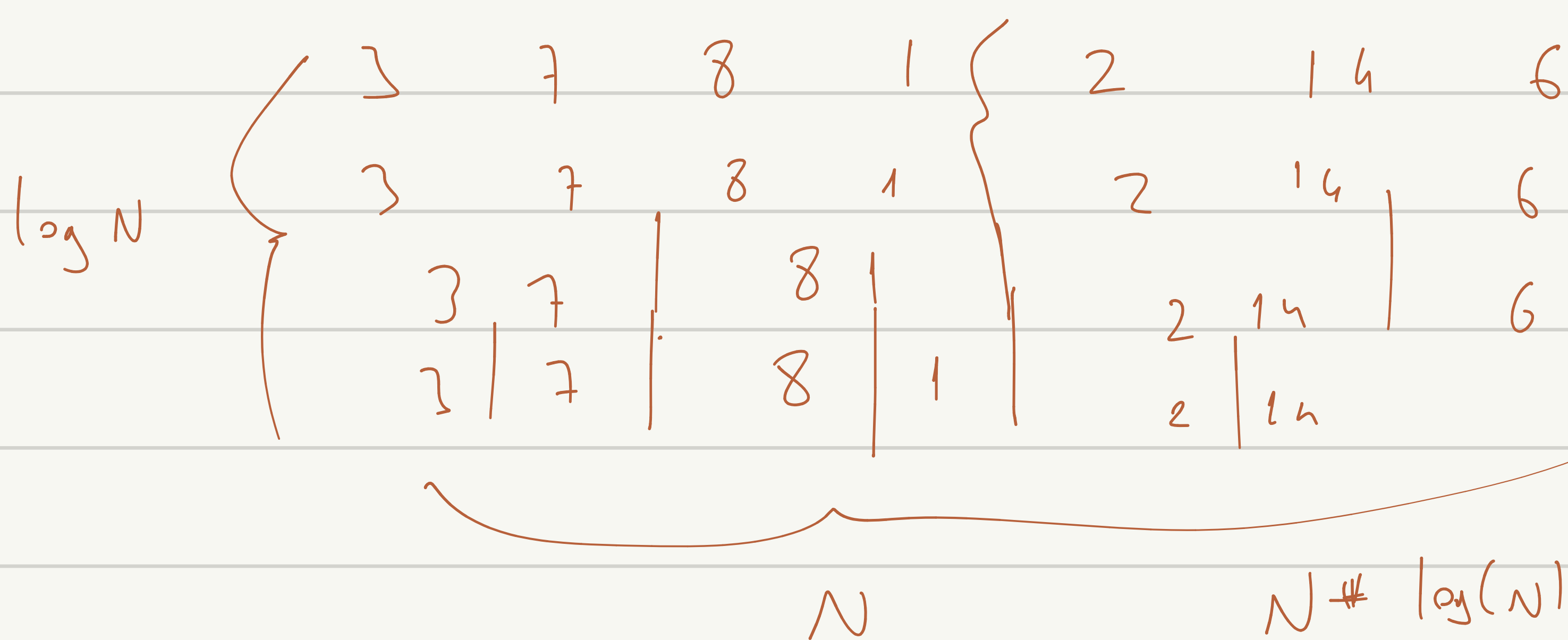
10^8 karşılaştırmalı saniye yapılan bilgisayarlar

n $O(n^2)$	1000 anlık	1 milyon 2.8 saat	1 milyar 317 yıl
merge sort $O(n \log n)$	anlık	1 saniye	18 dakika
Quick sort $O(n \log n)$	anlık	0.6 saniye	12 dakika

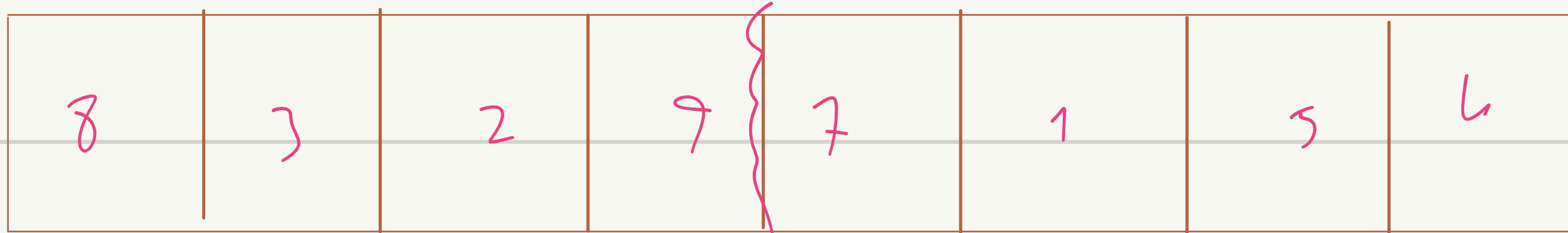
Divide and conquer



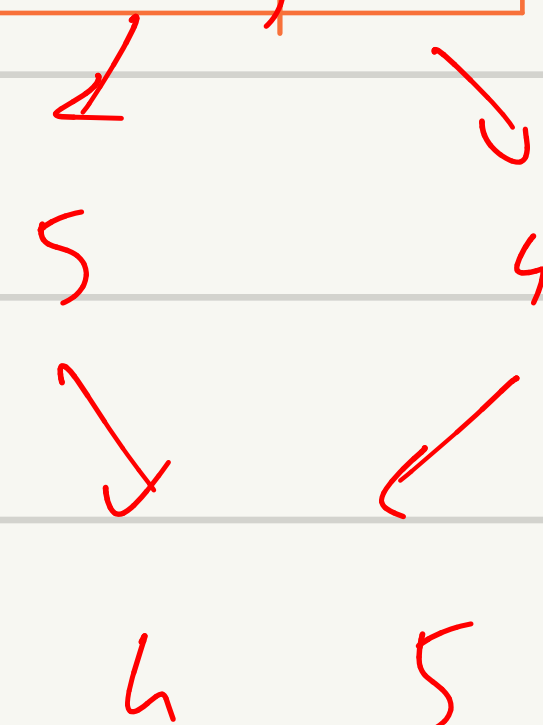
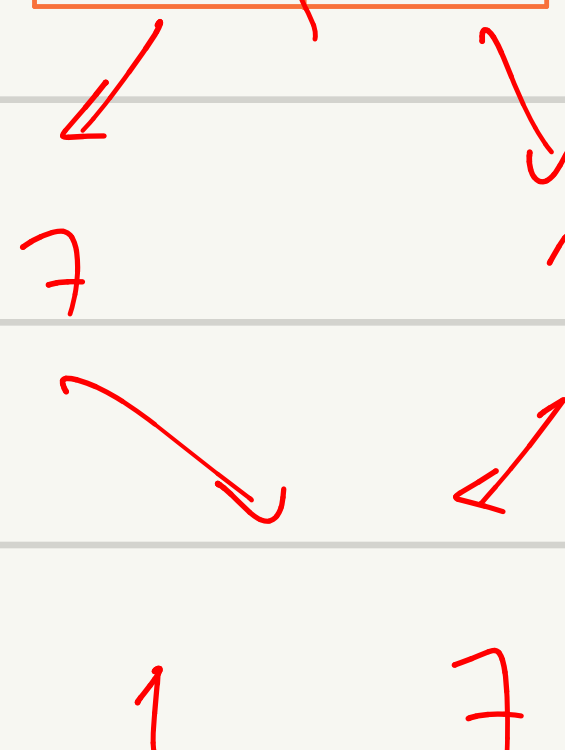
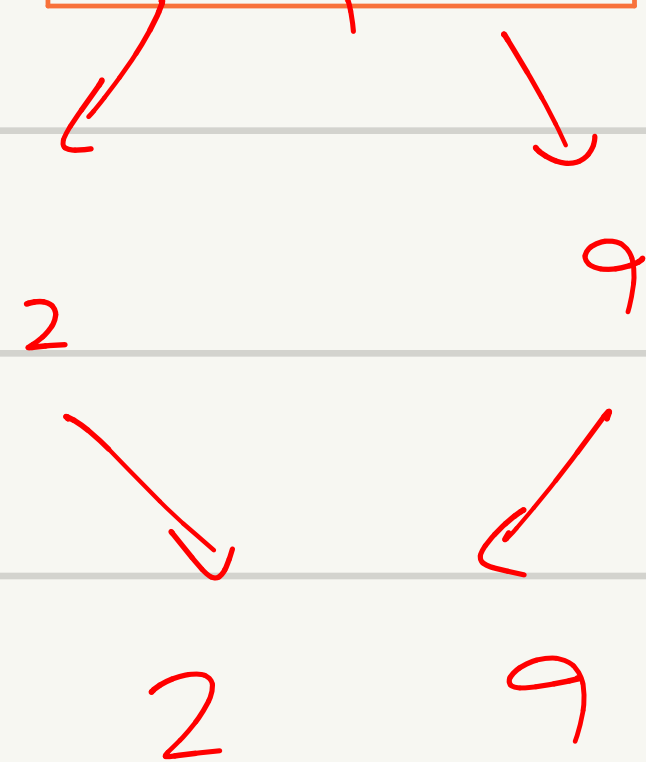
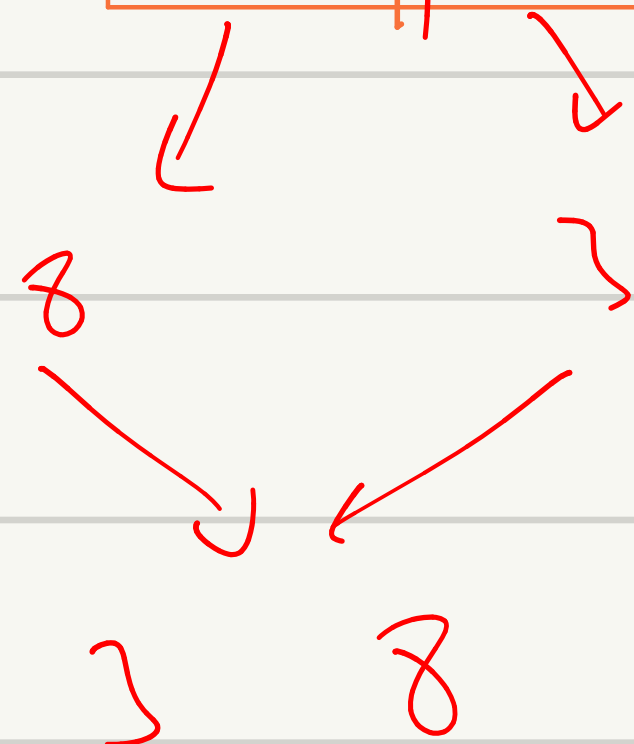
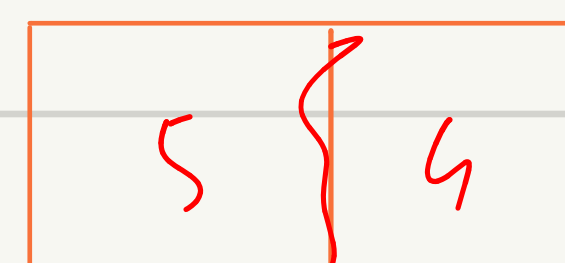
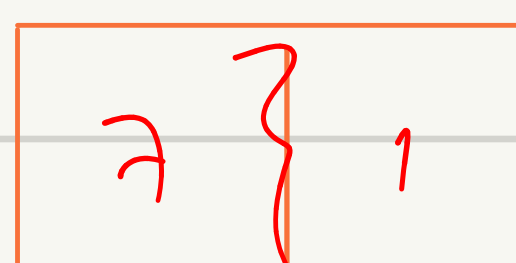
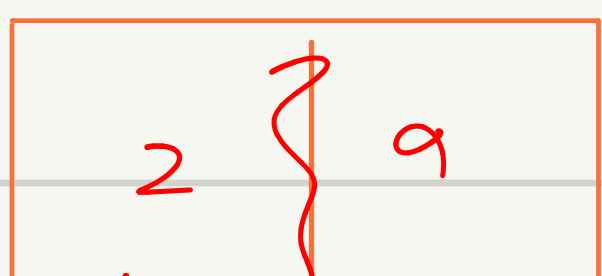
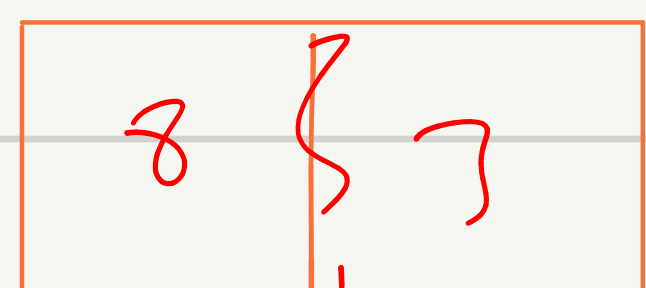
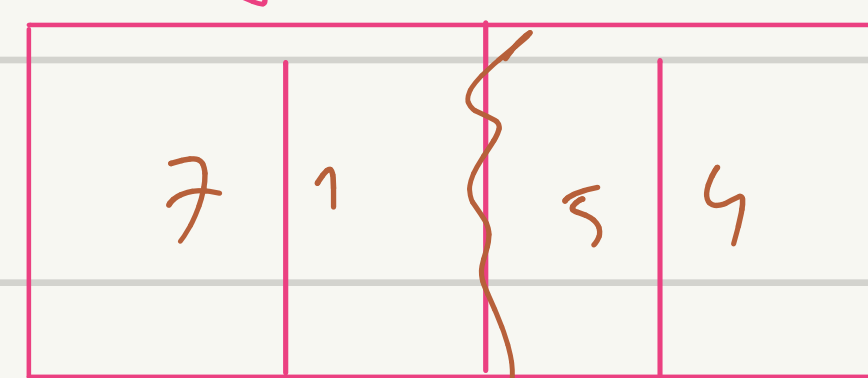
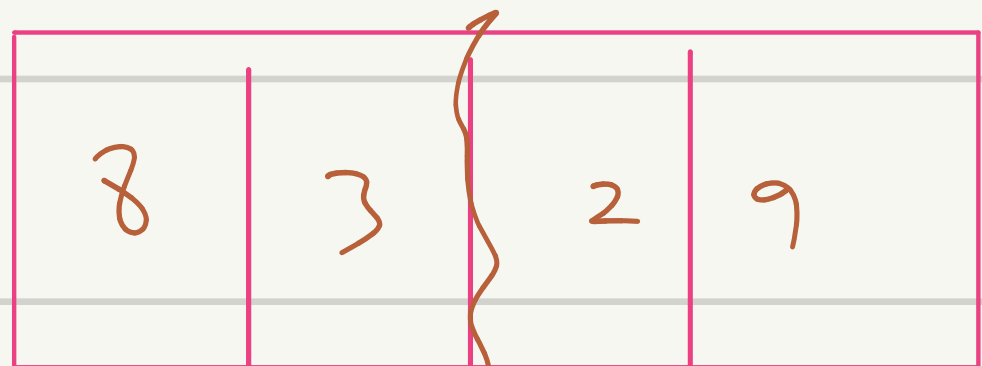
→ B'leri kontrolle üst üste eklemek merge ve $O(n^2)$ 'de olur.
Merge sort bu mantıkla en düşük parçalara bölüp merge ederim.



2



SPLIT



2 3 8 9

1 4 5 7

1 2 3 4 5 7 8 9

DIVIDE and Conquer

1: DIVIDE → Aynı algoritmayı uygulayabildiğimiz küçük parçaları bul

2: Bölünmüş parçaları işle

3: " " Birleştir

Merge Sort: 1-) SPLIT the array 2-) Sort each subsequence

3-) merge the two sorted subsequence

$A[1 \dots n-1]$ → $A[1 \dots n/2]$...
 $A[n/2 \dots n-1]$

```

void mergeSort(int *A, int p, int r) {
    // base case
    // son
    P = 0, r = 7, q = 3

```

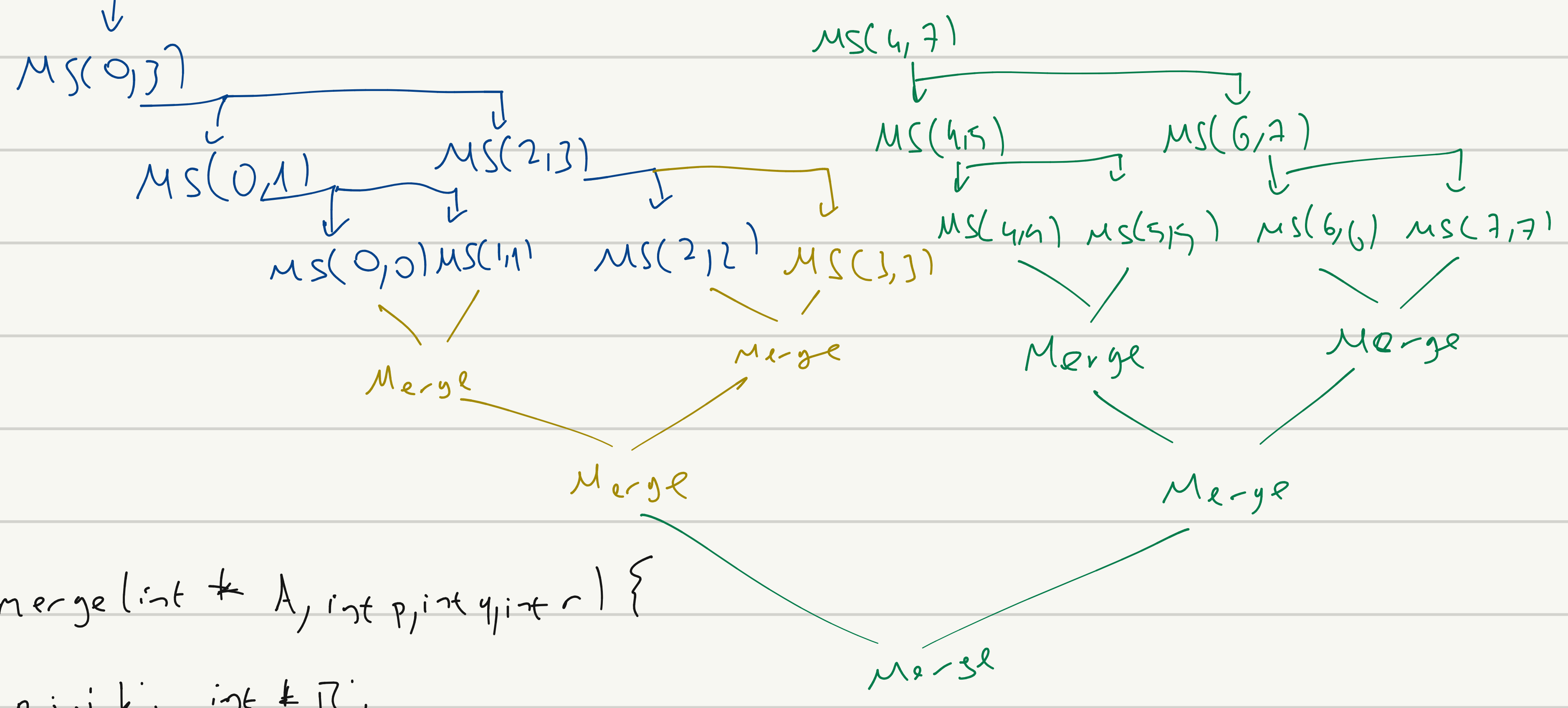
```

    if (p < r) {
        int q = (p+r)/2;
        mergeSort(A, p, q);
        mergeSort(A, q+1, r);
        merge(A, p, q, r);
    }
}

```

- MS(0,3)
- MS(0,1)
- MS(0,0)
- MS(1,1)
- MS(2,3)
- MS(2,2)
- MS(3,3)
- MS(4,7)
- MS(4,5)
- MS(4,4)
- MS(5,5)
- MS(6,7)
- MS(6,6)
- MS(7,7)

MS(0,7)



```

void merge(int *A, int p, int q, int r) {
    int n, i, j, k;
    int *B;
    n = r - p + 1; // element sayisi
    B = (int *) malloc(n * sizeof(int));
    i = p; j = q+1; k = 0;
    while (i <= q and j <= r) {
        if (A[i] < A[j])
            B[k++] = A[i++];
        else
            B[k++] = A[j++];
    }
}

```

```

while (i <= q) B[k++] = A[i++];
while (j <= r) B[k++] = A[j++];
for (i = p, k = 0; i <= r; i++, k++)
    A[i] = B[k];
free(B);
}

```

Recurrence Relation of Merge Sort

$$T(n) = \begin{cases} O(1) & n=0,1 \\ 2T(n/2) + n & \text{if } n \geq 1 \end{cases}$$

$\therefore \rightarrow O(\log_2 n * n) \rightarrow \Theta(\log_2 n * n)$

Exber bilgi

Genel böl ve yönetimin rekürrens banyamları

$T(n) = a * T(\frac{n}{b}) + f(n)$ *ifn sayıları* *is. yitir*
Zikacı kökük
ve $T(1) = c$

If $f(n) \in \Theta(n^\alpha)$ when $\alpha \geq 0$ $\in \log T(n)$

$T(n) = T(\frac{2}{3}n) + 1$
 $\alpha=0 \quad a=1 \quad b=\frac{3}{2}$
 $1^2 = (\frac{3}{2})^0 \rightarrow \Theta(\log_2 n * n^0) = \Theta(\log_2 n)$

Backward Substitution

$$T(n) = 2 * T(n/2) + n$$

$$T(n/2) = 2 * T(n/4) + n/2$$

$$T(n/4) = 2 * T(n/8) + n/4$$

$$T(n/2) = 4 * T(n/8) + n$$

$$T(n) = 8 * T(n/8) + 3n$$

$$= 2^i * T(\frac{n}{2^i}) + i * n$$

$$n = 2^i = n * T(1) + \log(n) * n$$

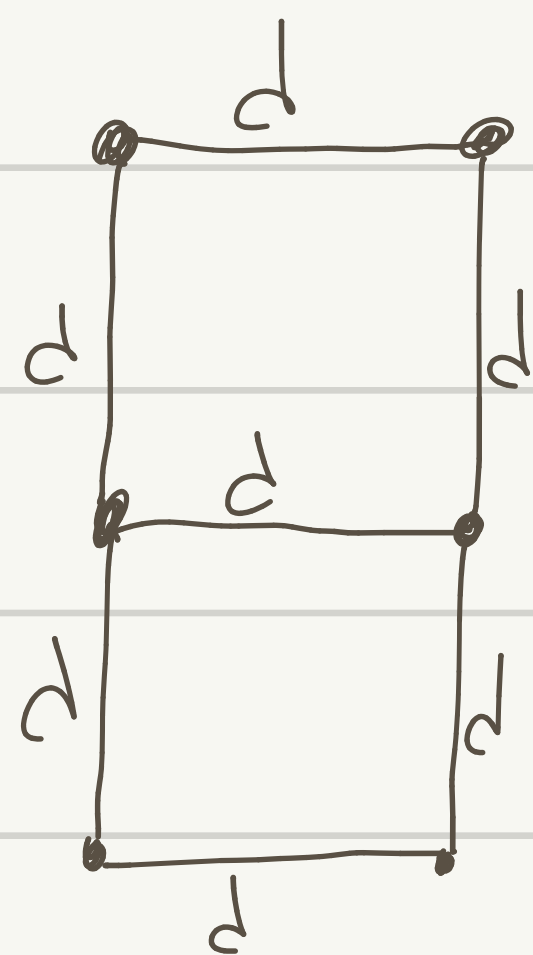
$$= n + n * \log(n)$$

$$\begin{cases} \Theta(n^a) & \text{if } a < b^a \\ \Theta(n^a \log n) & \text{if } a = b^a \\ \Theta(n^{\log_b a}) & \text{if } a > b^a \end{cases}$$

not: Ω ve O da da olur.

LAD 1

n elemanlı dizi var ve en yakın 2 elemanı buluyor



→ noktalar y'lerine göre sıralı
bundan dolayı $2d + d$ ile bir
alında en yakın 6 elemanı
buluruz. Örnekle kurgularız.

$$T(n) = 2T(n/2) + 6N$$

$$a=2 \quad b=2 \quad \alpha=1 \quad a=b^{\alpha} \quad n \log n$$

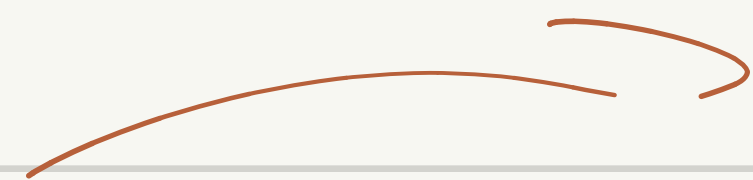
$$n < 3$$



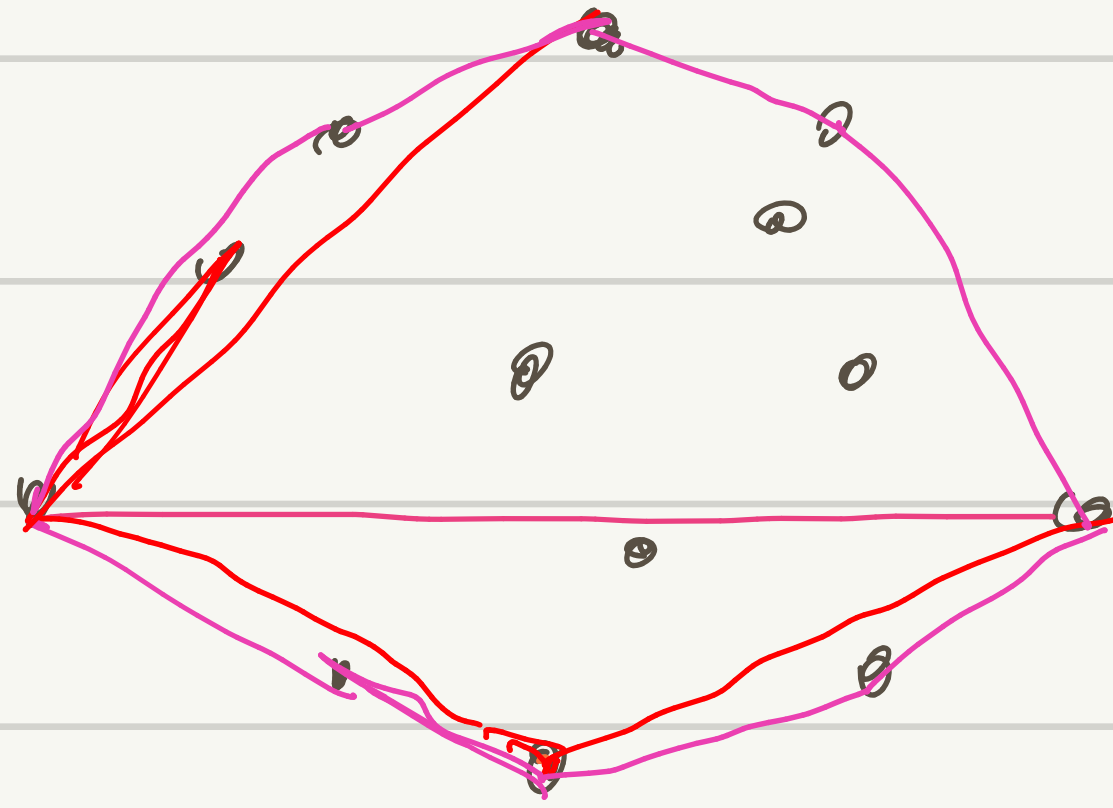
noktalar kümesindeki kapsama en küçük dış bölgeler

brute force

$$\binom{n}{3} \binom{n}{4} \binom{n}{5} \binom{n}{6} \dots \binom{n}{n} = 2^n$$



X eksenine göre en uzun 2 noktadan böleriz, sonra
n defa deneyerek 2 noktaya en uzak noktayı
seçiyoruz. Eğer dışta kalan varsa başka nokta seçiyoruz.



$$T(N) = 2T(N/2) + N$$
$$= n \log n \rightarrow \text{ortak } n$$

Kapitel 5

Quick Sort $\rightarrow$ In Place

	0	1	2	3	4	5	6	7
A	5	3	1	9	8	2	4	7

Pivot

$A[0 \dots s-1]$ pivot $A[s+1 \dots n-1]$
 $A < \text{pivot}$ $\text{pivot} < A$

3 2 1 4 7 11 13 8 12

	0	1	2	3	4	5	6	7
A	5	3	1	9	8	2	4	7

	0	1	2	3	4	5	6	7
A	5	3	1	4	8	2	9	7

	0	1	2	3	4	5	6	7
A	5	3	1	4	2	8	9	7

	0	1	2	3	4	5	6	7
A	2	3	1	4	5	8	9	7

$L=0$ $R=7$
 $S=4$ pivot=4

$L=0$ $R=3$ $S=1$
 $S=1$ pivot=2

$L=5$ $R=7$
 $S=6$ pivot=8

	0	1	2	3
A	2	3	1	4

	5	6	7
A	8	9	7

	0	1	2	3
A	2	1	3	4

	5	6	7
A	8	7	9

	0	1	2	3
A	1	2	3	4

	5	6	7
A	7	8	9

$L=0$ $R=0$
 ✗

$L=2$ $R=3$
 $S=2$ pivot=2

$L=2$ $R=1$
 ✗

$L=3$ $R=3$
 ✓

	0	1	2	3
A	2	3	1	4

$L=2$ $R=1$
 ✓

$L=3$ $R=3$
 ✓

PARTITIONING

```
quicksort(n,A[l....r]&{
  if(l<r&{
    s=partition(A[l...r]&; //Pivot yerine yerleşir küçükler
    solunda büyükler sağında kalır ÖDEV
    quicksort(A[l...s-1]&;
    quicksort(A[s+1.....r]&;}}
```

Best Case $\rightarrow T(n) = 2T(n/2) + n$ $T(1) = 0$

$a=2 \quad b=2 \quad d=1$

$2 \cdot 2^1$ olduğu için $n^d \log n \rightarrow n \log n$

Pivot ortu olduğun best case her aynı olduğundan $\Theta(n \log n)$

Worst Case

küçükten büyüğe-büyükten küçüğe sıralı oluncası

$T(n) = T(n-1) + n$ for $(n \geq 1)$ $\Theta(n^2)$

$T(0) = T(1) = 1$

Average

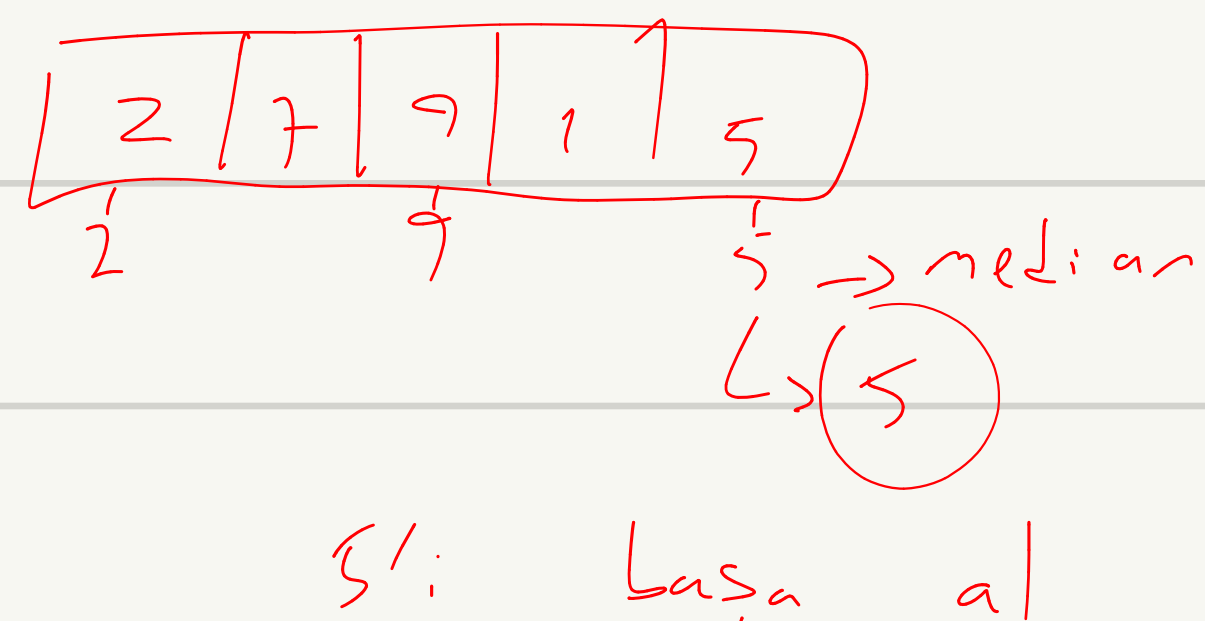
Position of pivot	left of pivot	right of p.	$T(n_{left})$	$T(n_{right})$
0	0	$n-1$	$T(0)$	$T(n-1)$
1	1	$n-2$	$T(1)$	$T(n-2)$
2	2	$n-3$	$T(2)$	$T(n-3)$
⋮				
$n-1$	$n-1$	0	$T(n-1)$	$T(0)$

$T(n) = \frac{1}{N} \cdot \sum_{k=1}^N (T(k-1) + T(n-k) + n) \rightarrow T(n) = 1,39 N \log_2(N) \in O(n \log_2 n)$

$\hookrightarrow$ QS merge S.'den %39 fazla kıyaslama yapıyor ama elemanın değişimi u2
onun dolayısı daha hızlı.

worst case'den kurtulma yolları

- ① karıştırmak
- ② median of samples $\rightarrow$
- ③ random pick pivot



Decrease And Conquer

- ① Decrease by a constant factor

binary search

problem of size n

$\downarrow$
" " " " $n/2$

$\vdots$
solution to the original problem

$$T(n) = T(n/2) + c$$

- ② Variable size decrease

$$\text{GCD}(a, b) = \begin{cases} a & \text{if } b = 0 \\ \text{GCD}(b, a \bmod b) \end{cases}$$

$$\text{GCD}(80, 44)$$

$$\hookrightarrow \text{GCD}(44, 36)$$

$$\hookrightarrow \text{GCD}(36, 8)$$

$$\hookrightarrow \text{GCD}(8, 4)$$

$$\hookrightarrow \text{GCD}(4, 0)$$

$$\hookrightarrow 4$$

- ③ Decrease by a constant topological sort

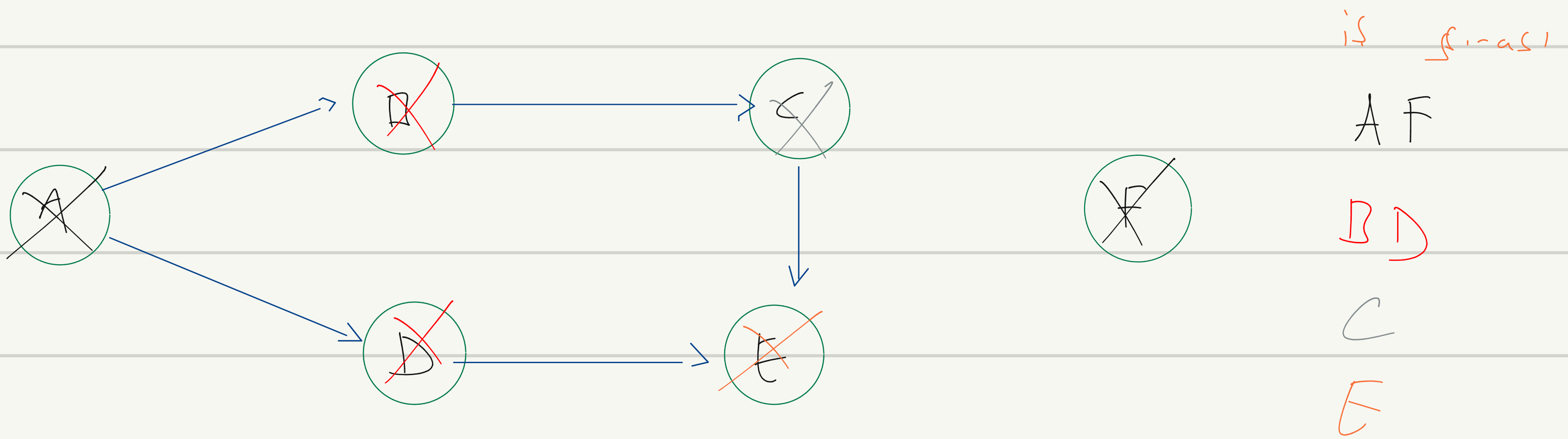
6. Hafta

Decrease and conquer

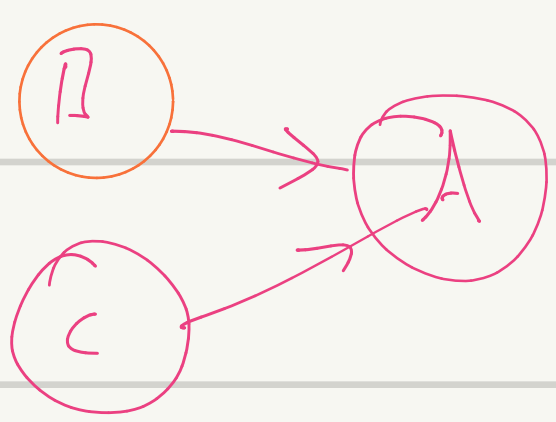
Decrease by a constant

Topological sort $\rightarrow$ Graflar $\rightarrow$ çizilir

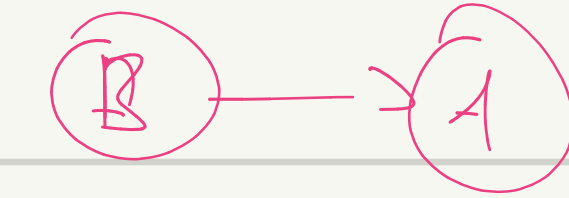
Diğerleri olmalı, önceki olan şartlar verildi



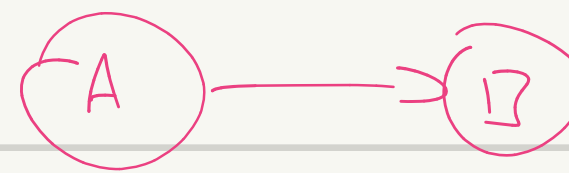
In-degree



A'nın In-degreesi 2



A'nın In-degreesi 1



A'nın In-degreesi 0

Konşuluk Listesi

in-degree

0	A $\rightarrow$ B $\rightarrow$ D
1	B $\rightarrow$ C
1	C $\rightarrow$ E
1	D $\rightarrow$ E
2	E $\rightarrow$ Null
0	F $\rightarrow$ Null

$\rightarrow$ in-degree sayısı bulma

Listeyi geziyoruz, nextlediğimiz neye eşitse onu artırıyoruz

$\rightarrow$ Bu bir işin yapılabilmesi için baş işin bitmesi

gerektiğini işleyemeyiz $\rightarrow$ E için önce A, sonra B, sonra C'nin bitmesi lazım. Ama $in-degree[E] = 2$

$\rightarrow$ A bitince B ve C'yi azaltmak lazım. $in-degree[B] = 1$

Hangi işten başlayacağımızı bulmak için queue'ya

atıyoruz.

A | F

0	A → B → C → D
1	B → C
2	C → F
1	D → E
2	E → Null
0	F → Null

F | D | B

0	A → B → C → D
0	B → C
1	C → F
0	D → E
2	E → Null
0	F → Null

D | B

0	A → B → C → D
0	B → C
1	C → F
0	D → E
2	E → Null
0	F → Null

B

0	A → B → C → D
0	B → C
1	C → F
0	D → E
1	E → Null
0	F → Null

Queue

C

0	A → B → C → D
0	B → C
0	C → F
0	D → E
1	E → Null
0	F → Null

E

0	A → B → C → D
0	B → C
0	C → F
0	D → E
0	E → Null
0	F → Null

① Store each vertex's in-degree in array $O(E)$

② While there are vertices remaining:

find a vertex with in-degree zero $O(V)$

reduce in-degree of all vertices adjacent to it by 1 $O(E)$

mark in-vertices in-degree -1

Queue $\rightarrow O(V+E)$

① Store each vertex's in-degree in array $O(E)$

② Initialize a queue with all in-degree zero vertices $O(V)$

③ While there are vertices remaining in the queue

- Dequeue and output $O(V)$

- Reduce in-degree of all vertices adjacent to it by 1 $O(E)$

- Enqueue vertices with in-degree zero $O(E)$

	Worst			Average		
	search	insert	delete	search	insert	delete
Lineer diz.	n	1	n	n/2	1	n/2
Siral. diz.	$\log_2 n$	n	n	$\log_2 n$	n/2	n/2
Dengesiz Tree	n	n	n	$\log_2 n$	$\log_2 n$	$\log_2 n$
Dengeli Tree	$\log_2 n$	$\log_2 n$	$\log_2 n$	$\log_2 n$	$\log_2 n$	$\log_2 n$

Hashing

0	Null
1	Null
2	Null
3	Null
4	18 247
5	Null
6	Null
7	Null
8	Null

hash table

18 247 → hash function → h

insert → O(1)

search O(1)

hash fun. ← 18247

① Hash tablo'nun uzunluğu ne olmalı? Aynı indise birden çok sayı atanarak için.

② Hash fonksiyonunun nasıl seçmeliyim?

③ Çatışma olursa ne olacak

Hash functions

① Division method

$$\text{hash function (key)} = \text{key} \bmod m$$

$$\text{key} = 25$$

$$\text{hash}(25) = 25 \% 7 \rightarrow 4$$

$$\text{hash}(7) = 7 \% 7$$

0
1
2
3
4
5
6

table size → 0 → n-1

asal sayı olursa daha iyi
2'nin katları olabilir.
Ama 7 olursa 7'nin katları az

② Multiplication method

$$\text{hash function (key)} = \left\lfloor m * (\text{key} * A) \bmod 1 \right\rfloor$$

↑
size

$$A = \frac{\sqrt{5}-1}{2} = 0,618033$$

key 123.456

$$\text{hash function}(123.456) = \left\lfloor 10.000 * \left[(123.456 * 0.618033) \bmod 1 \right] \right\rfloor$$

$$123.456 * 0.618033 \bmod 1 = 0.0041151$$

$$10.000 * 0.0041151$$

$$41 \quad \boxed{123.456} \quad \left\lfloor 41.151 \right\rfloor = 41$$

HAFTA 7

Hashing

- 1-) Division method $hash(key) = key \% m$ table size
- 2-) Mult. " $hash(key) = \lfloor m * (key \% 1) \rfloor$

→ tablo ma
Collision problemi

① Open Addressing

key = 5
 $hash(key) = 5 \% 11$

0 ... m-1

0	1	2	3	4	5	6	7	8	9	10
					5	16			9	

key = 9
 $hash(key) = 9 \% 11$

key = 16
 $hash(key) = 16 \% 11 = 5$???

1.1 Linear Probing → each check is a probe

$$hash(key, i) = [hash(key) + i] \% m$$

key = 16 $i = 0 \rightarrow hash(16, 0) = 5$

key = 16 $i = 1 \rightarrow hash(16, 1) = (5 + 1) \% 11 = 6$

Arama işleminde Null görüne kadar ya da başa dönerne kadar uğruyoruz. $Search(key, i)$

başla dönerne kadar
B. kelenmez
 $M \neq N$ olur

while ($hash[adr] \neq Null$ && $i < m$ && $hash[adr] \neq key$) { }

Linear Probing'in Clustering problemi var → Mesela 5 5'e geldi

16 da 6'ya geldi. Bu örnek için 6 asıl yerine gelecek. Bunden dolayı

1.2 Double Hashing

$$hash(k, i) = [hash1(k) + i * \underbrace{hash2(k)}_{\text{jump size}}] \% m$$

k = 14 $hash(14) = 1$

k = 79 $hash(79) = 1 \rightarrow hash(79, 1) = (1 + 1 * 2) \% 13 = 3$

Bunlar başta doluyor

m = 13

0	1	2	3	4	5	6	7	8	9	10	11	12
0	14		79									12

$$hash1(key) = key \% 13$$

$$hash2(key) = key \% 11$$

Birthday paradox

2 kişinin doğum gününün farklı olma olasılığı

$$\frac{365}{365} \times \frac{364}{365}$$

k kişi $\rightarrow \sum_{i=0}^{k-1} \frac{365-i}{365}$ $\rightarrow$ Table size m $k=2 \rightarrow \sum_{i=0}^{k-1} \frac{m-i}{m}$

taylor expansion; yaklaşık olarak $e^{-\frac{(k-1) \cdot k}{2n}} = p$

collision problem = $1 - e^{-\frac{(k-1) \cdot k}{2n}}$
↓
çakışma olasılığı

load factor

$\frac{N}{M}$ $\frac{10}{100} \rightarrow \%10$

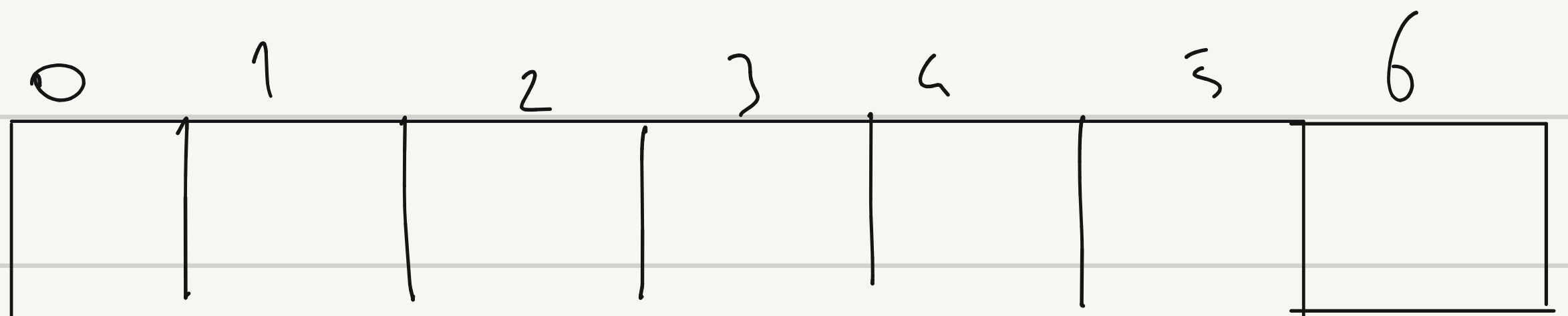
Open addressingde load factor $\rightarrow$ yaklaşık %50

$M \approx 2N$

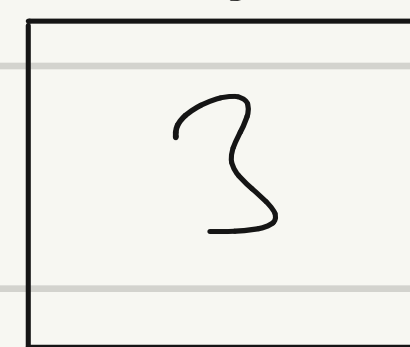
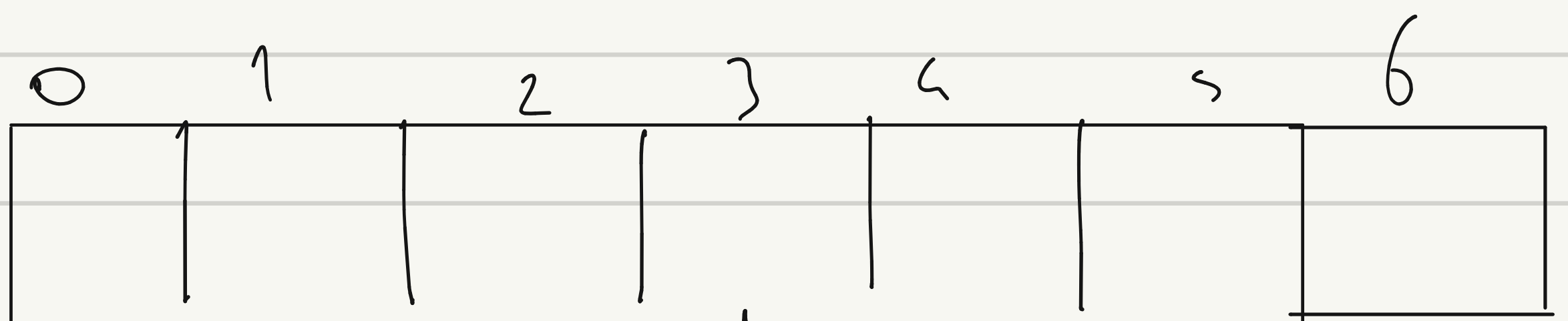
② Separate Chaining

$$M \approx N/5$$

load factor $\approx \%20$

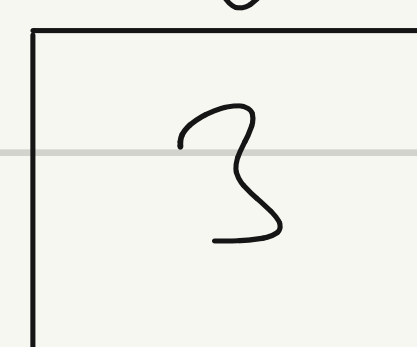
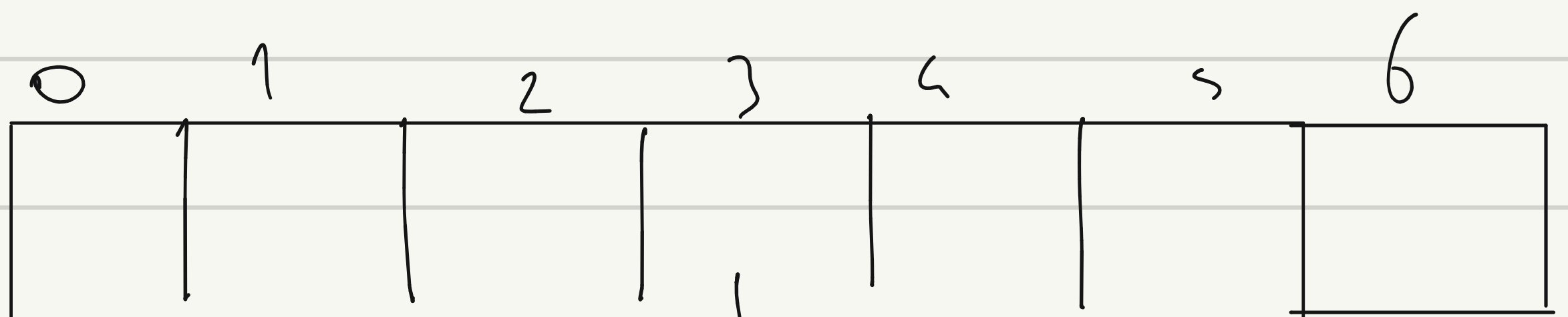


key = 3 $3 \% 7 = 3$



→ ilk elemana ekleniyor

key = 10 $10 \% 7 = 3$



```

struct node {
    int key;
    char name[50];
    struct node * next;
};
    
```

key unique olmalı,

① key Stringse

"Kasra" hash("kasra")

70 10605010
↳ 1,1,1,1,1,1 toplam 200 olur ama aryan'ın da değeri 200 olur.

tercih edilmez

bu

Length = L $\rightarrow k^{L-1} \cdot \text{ser}[0] + k^{L-2} \cdot \text{ser}[1] + \dots + k^0 \cdot \text{ser}[L-1]$ ile çözeriz.

k asal ve küçük olmalı..

Horner's method k=31

hashkey = 0

length = strlen(s);

for(i=0; i < length; i++) {

temp = s[i] - 'A' + 1;

hashkey = k * hashkey + temp;

Universal hashing $\rightarrow$ saldırıların fonksiyonunu bilerek collision oluşturmaya çalışıyor.

Birden fazla hash fonksiyonu kullanılır ve rastgele seçilir, k-ş. taraf

bu anlattığımız işin saldırması zor

$\text{hash}(\text{key}) = [(a * k + b) \bmod p] \bmod m$

a, b ve p değişken

Hash tablosunun eleman eklenmesi, eğer varsa eklenmeyen - yoksa eklenen

algoritmayı tasarla

↓
i = 0
adr = key % m

while (hash[adr] != NULL && i < m && hash[adr] != key) {

i++;
adr = (adr + 1) % m; }

if (i == m)

printf("tablo dolu")

else if (hash[adr] == NULL)

hash[adr] = key;

else

printf("eleman zaten var");

LAB 2

11

$$\sum_{i=1}^n \sum_{j=1}^n ij \rightarrow \sum_{i=1}^n i \sum_{j=1}^n j \rightarrow \sum_{i=1}^n i \frac{n(n+1)}{2} \rightarrow \left(\frac{n(n+1)}{2} \right)^2$$

$$\textcircled{2} \sum_{i=1}^n i^3 \rightarrow \left. \begin{array}{l} T(N) = T(N-1) + 3 \\ T(N-1) = T(N-2) + 3 \\ T(N-2) = T(N-3) + 3 \\ T(1) = 1 \end{array} \right\} \begin{array}{l} T(N) = T(n-i) + 3 \times i \quad N \geq 1 \\ T(N) = 1 \quad i = 1 \\ i = n-1 \quad i = 1 \end{array}$$

$$T(N) = T(1) + 3(n-1) \rightarrow \underline{3n-2}$$

Backward

forward

n	$T(n)$
1	1
2	4
3	7
4	10
$\vdots$	
N	$T(n-1) + 3$

$\rightarrow 3n-2$

$$\textcircled{3} Q(n) \quad n^2 \text{ algo}$$

if $(n=1)$ return 1, $\rightarrow$ Backward

return $Q(n-1) + 2 \times n - 1$

$$\rightarrow T(n) = T(n-1) + 2$$

$$T(n) = T(n-2) + 2 + 2$$

$$T(n) = T(n-1) + 2$$

$$T(n-1) = T(n-2) + 2$$

$\vdots$

$\rightarrow 2n-1$

$$T(2) = T(1) + 2 \rightarrow 3$$

$$T(1) = 1 \rightarrow 1$$

$$T(n) = T(n-i) + 2 \times i$$

$$i = n-1$$

$$T(n) = T(1) + 2 \times (n-1)$$

$$= 1 + 2n - 2 = 2n - 1$$

4 Selection sort rekursif $\rightarrow T(n) = T(n-1) + n$

$$T(n-1) = T(n-2) + n-1$$

$$T(n-2) = T(n-3) + n-2 \quad \rightarrow \quad T(n) = T(n-i) + n-i+1 + n-i+2 + n-i+3 \dots n$$

$$T(2) = T(1) + 2$$

$$j = n - 1$$

$$T(1) = 1$$

$$T(n) = T(1) + 2 + \dots + n \rightarrow \frac{n(n+1)}{2}$$

Backward

$$T(n-1) = T(n-2) + n-1$$

$$T(n) = 1 + 2 + 3 \dots$$

$$T(2) = 1 + 2$$

$$T(1) = 1$$

5 inversion problem

$$\text{diz:}[i:] \supset \text{diz:}[j:]$$
$$re \quad i < j \quad isl$$

inversion var direkt. Bunun nasıl bulabiliriz?

1 20 8 6 | 7 9 5 6 2

1 2 3 4 5 6

1 | 2 3 | 6 → counter = 1
 ↑ ↑ 8 6 inversion
 1 2 → inversion ~?

Buras kurisi
Diger sayfa
Daha iyi

1 20 8 6 | 7 9 5 6 2

[illegible]

↑ geben - geben - gegeben

1 - 8 ✓

$$22 - 8 = 14$$

22 - 6 count ++

yukarı çıkarken sıralandığı için mesela 5 7 9 ve 2 6 var. 0 zaman 5 ile 2'yi karşılaştırırız ve count'u direkt 3 artırırız. Çünkü 5 sıkıntıysa 7 ve 9'da sıkıntı. Sonra 5-6'da yok. Sonra 6-7'yi karşılaşıp 2 artırırız. Çünkü 7 ile varsa 9 ile de vardır.

1 20 8 6 | 7 9 5 6 2

1. adım

1 20 | 8 6

1 20

1 20 8 6 | 7 9 5 6 2

2. adım

1 20 | 8 6

count = 0 | 8 6 → count = 1

not soldaki 6 karşılaştı

1 20 8 6 | 7 9 5 6 2

1 20 | 6 8 → 1 < 6 ✓ 20 > 6 count ++

count = 0 | count = 1 20 > 8 count ++

1 6 8 20 | 7 9 | 5 6 2

count = 3 7 | 9 5 6 2

count = 0 | count = 1 7 9

1 6 8 20 | 7 9 | 5 6 2

count = 3 7 | 9 5 | 6 2

count = 0 | count = 1 count = 0 count = 0

1 6 8 20 | 7 9 | 5 6 2
 count = 3
 count = 0 | count = 1
 count = 0
 count = 0
 6 2
 count = 1

1 6 8 20 | 7 9 | 5 6 2
 count = 3
 count = 0 | count = 1
 count = 0
 count = 0
 count = 1

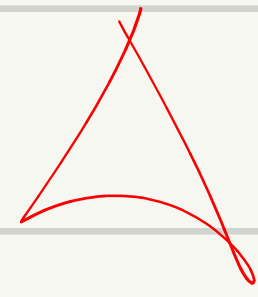
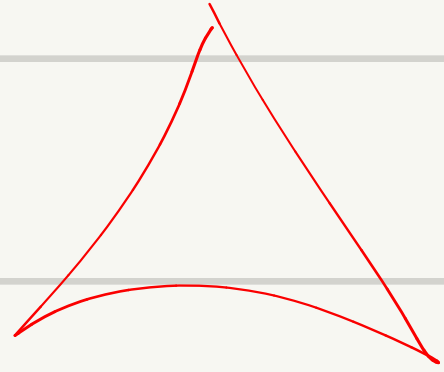
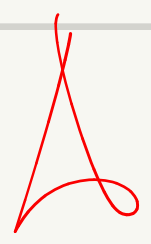
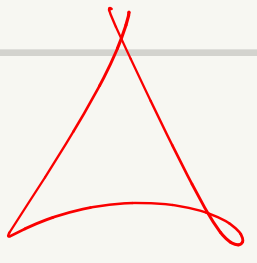
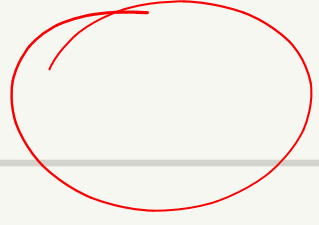
1 6 8 20 | 7 9 | 5 6 2
 count = 3
 count = 0 | count = 1
 count = 0
 count = 0
 count = 1
 5 - 6 ✓
 5 - 2 count++

1 6 8 20 | 7 9 | 2 5 6
 count = 3
 count = 0 | count = 1
 count = 0
 count = 2 → 7 - 2
 count = count + 2
 7 - 5
 7 - 6
 count = count + 2
 count = count + 2

1 6 8 20 | 2 7 6 7 9
 count = 3
 count = 7 → 1 - 7 ✓
 6 - 2 count = count + 3
 6 - 5 count = count + 3
 6 - 6 ✓
 8 - 6 count = count + 2
 8 - 7 " "
 8 - 9 ✓
 20 - 9 count++
 10 + 11 = 22
 soldat: 6

(6) Vida Somon

→ hangi vida hangi somona



vidaları arasında kıyaslamak yet. somonları da

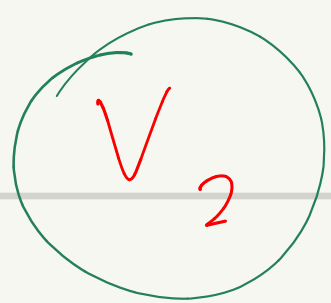
burada bir vidayı pivot seçeriz ve Quick sort gibi yaparız.
↳ somonlar için vidaları vidalar için somonları

S₁

S₃

S₄

S₂



S₁

S₂

S₄

S₃

V₄

V₂

V₃

V₁

S₁

S₂

S₄

S₃

V₁

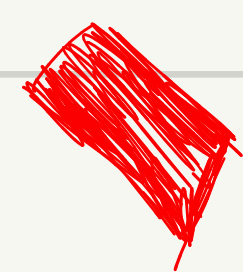
V₂

V₃

V₄

→ Burası için tekerrür

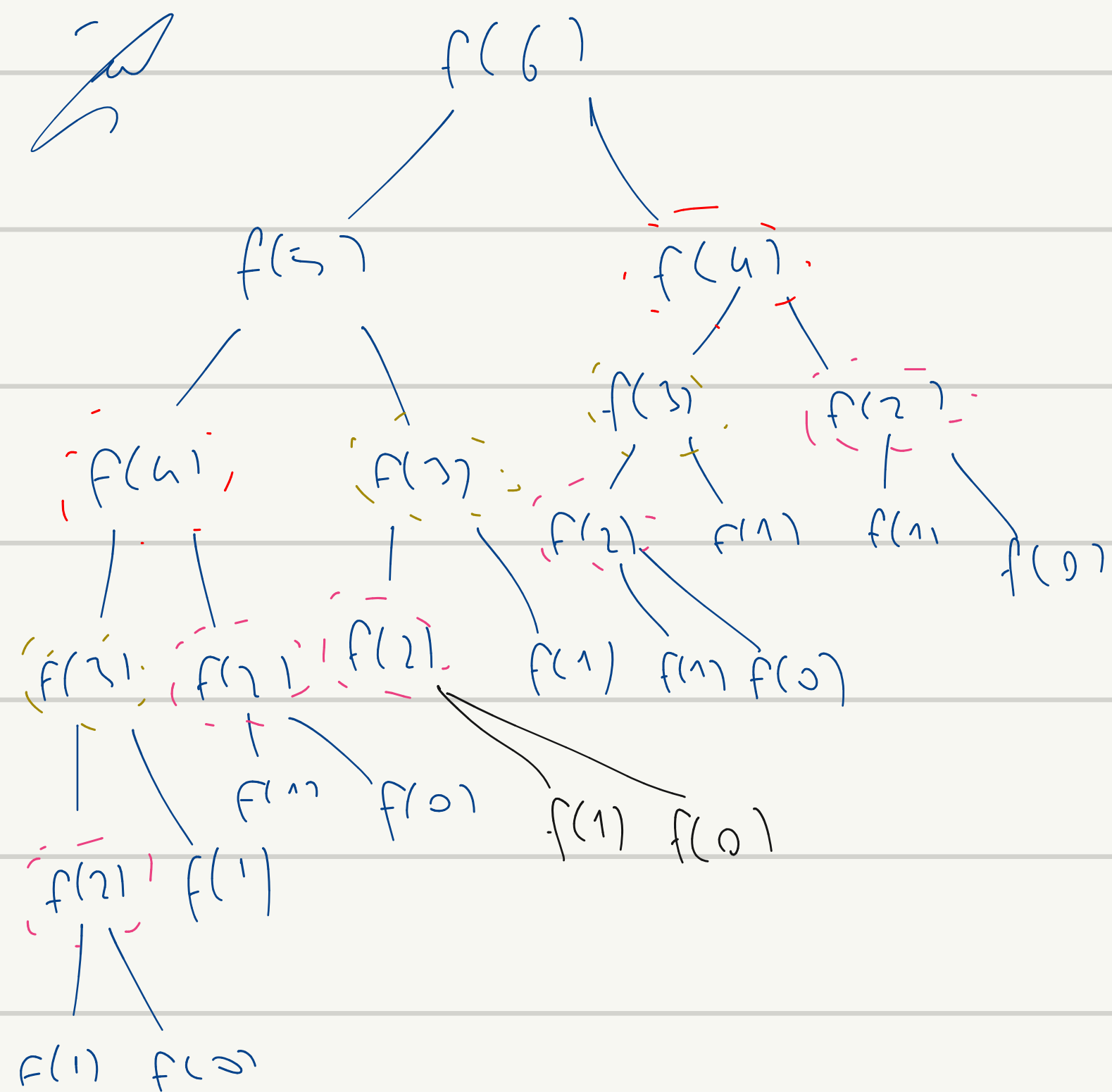
↳ Burası Bitti!



Dynamic Programming

Fibonacci numbers

0 1 1 2 3 5 ...



```

int fibonacci(int n)

```

```

{ if (n == 0)
  return 0;

```

```

  if (n == 1)
    return 1;

```

```

  return fibonacci(n-1) + fibonacci(n-2);
}

```

Minimum coin change problem

coins = {1, 2, 5} value = 11 : coin Answer = 5+5+1

goal 1 1+1+1+...+1

1+1+1+...+2

1+1+1+...+2+2

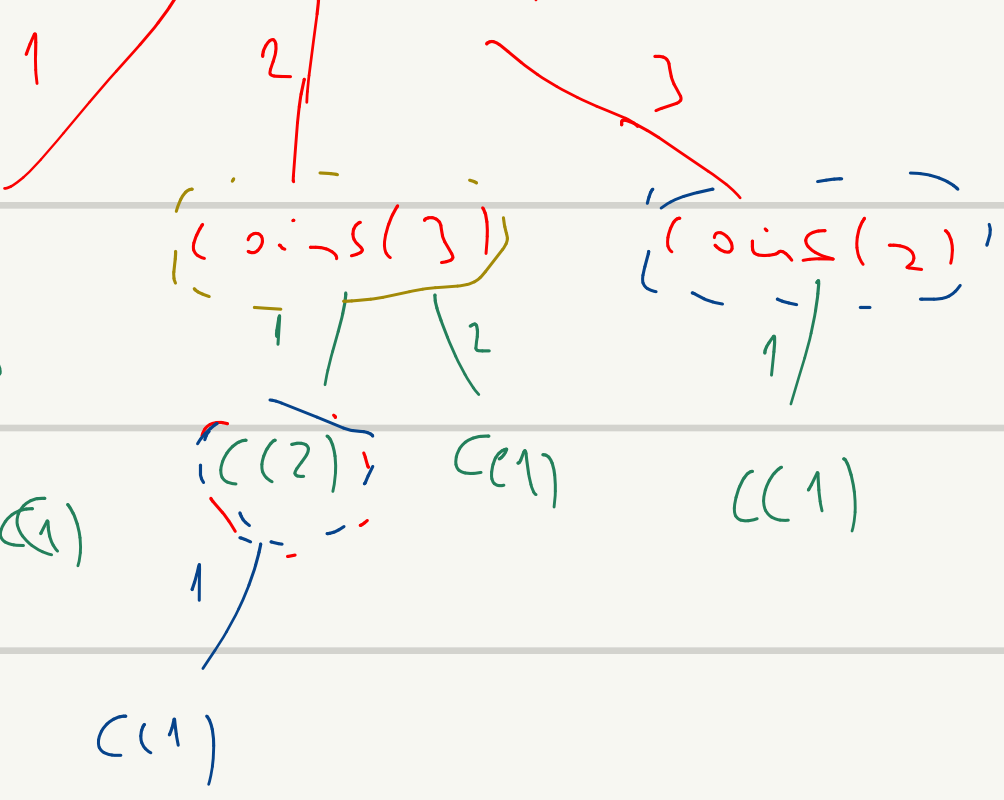
;

5+5+1 → Exhaustive Search

→ Exponential arbeiten

Run time: da nicht möglich $P(11) = P(10) + 1$ gibt

Coins(5) → coins [1, 2, 5]



coins(1)

```

int minCoin(int* coins, int n, int value&{

```

```

  int minCount = MAX;

```

```

  if(value == 0&{

```

```

    return 0;

```

```

  for(i=0; i<n; i++){

```

```

    if(coins[i] <= value&{

```

```

      tmp = minCoin(coins, n, value-coins[i]&

```

```

      if(tmp < MAX && tmp+1 < minCount&

```

```

        minCount=tmp+1;
      }

```

```

  if(minCount == MAX&

```

```

    return -1;

```

```

  else

```

```

    return minCount;
}

```

→ $O(2^n)$ values

Dynamic Programming

memoization

coins = {1, 2, 5} value = 11 Answer = 5 + 5 + 1

0	1	2	3	4	5	→ Amount
0	∞	∞	∞	∞	∞	
∞	∞	∞	∞	∞	∞	→ minimum number of coins for each amount
6	7	8	9	10	11	→ Amount

ilk durum

1 TL

0	1	2	3	4	5
0	1	2	3	4	5
6	7	8	9	10	11
6	7	8	9	10	11

min(0+1, ∞)

min(10+1, ∞)

2 TL

0	1	2	3	4	5
0	1	1	2	2	3
3	4	4	5	5	6
6	7	8	9	10	11

min(dizi[1]+1, dizi[2]+dizi[1], dizi[3]) → ?

min(dizi[3]+1, dizi[4]+dizi[1], dizi[5]) → ?

5 TL

0	1	2	3	4	5
0	1	1	2	2	1
2	2	3	3	2	3
6	7	8	9	10	11

min(0+1, 3)

min(1+1, 3)

min(1+3, 3)

dizi[6] = min(dizi[1]+dizi[5], dizi[6])

dizi[7] = min(dizi[7], dizi[2]+1, dizi[6]+dizi[1])

→ PTL

C[P]: minimum number of coins

x: first coin

C[P] = 1 + C[P-x]

C[P] = min { C[P-di] + 1 if P > 0
 : di < P
 0 if P = 0 }

C[8] = min { C[8-3]
 C[8-1]
 C[8-5] }

$C[0] = 0;$

for $p=1$ to n do {

$min = \infty;$

for $i=1$ to k do {

if $d[i] \leq p$ then

if $(1 + C[p - d[i]]) < min$ then

$min = 1 + C[p - d[i]];$

$C[p] = min;$

}



Knapsack Problem

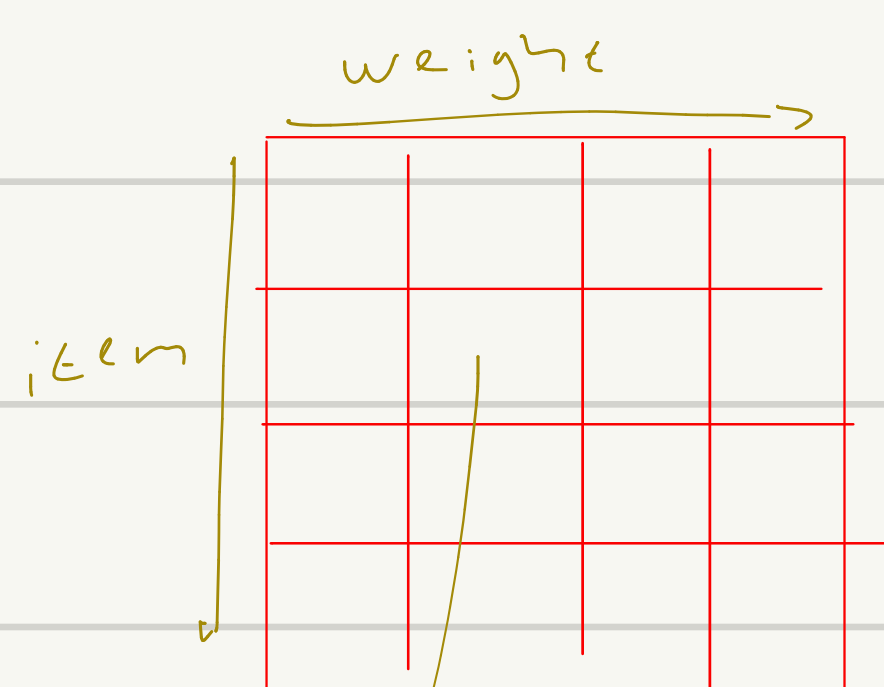
$W = \text{max weight}$

$N = \text{number of items}$

i^{th} item $\rightarrow$ value, weight

maximize $\sum V_i$ subject to W

Brute force 2^n



$\rightarrow$ maximum profit

from item 1 to item i

from weight 1 to weight j

item

weight

value

$W = 5 \text{ kg}$

1

2

12

2

1

10

3

3

20

4

2

15

Case 1: Thief takes item i

Case 2: Thief doesn't take item i

Rekursiyonun başlatılması

$P(i, w) = P(i-1, w)$

not include item

Before item i

0 kiloğa 2 kilo eklenmesi için

12 mi 10 nu büyük

item

	weight					
	0	1	2	3	4	5
0	0	0	0	0	0	0
1	0	0	12	12	12	12
2	0	10	12	22	22	22
3	0	10	12	22	30	32
4	0	10	15	25	30	37

10. Hufte

0-1 Knapsack Problem

$W = 11$ item $\begin{cases} \rightarrow \text{value} \\ \rightarrow \text{weight} \end{cases}$

item	value	weight
1	1	1
2	6	2
3	18	5
4	22	6
5	28	7

item $\downarrow$ $\begin{cases} \text{weight} \\ \text{value} \\ \text{Profit} \end{cases}$
 $P[i, w]$

	0	1	2	3	4	5	6	7	8	9	10	11
0	0	0	0	0	0	0	0	0	0	0	0	0
1	0	1	1	1	1	1	1	1	1	1	1	1
2	0	1	6	7	7	7	7	7	7	7	7	7
3	0	1	6	7	7	18	19	24	25	25	25	25
4	0	1	6	7	7	18	22	24	28	29	29	40
5	0	1	6	7	7	18	22	28	29	34	35	40

Case 1: Take item:

$$P(i, w) = P(i-1, w-w_i) + \text{value}_i$$

$\downarrow$ now $\downarrow$ weight $\downarrow$ column $\downarrow$ weight of item

Case 2: not take item:

$$P(i, w) = P(i-1, w) \quad \begin{cases} \rightarrow w_i > w \\ \rightarrow \text{Take max} \{v_i + P(i-1, w-w_i), P(i-1, w)\} \end{cases}$$

$P(5, 3) = P(4, 3) \rightarrow \text{weight}[5] > 3$

Angilini aldim?

$$P(5, 11) = P(4, 11)$$

al madim

$$\text{else aldim } P(4, 11-6) = P(2, 11-6)$$

$O(n \times w)$

$\begin{cases} \rightarrow \text{item} & \text{space} \\ \rightarrow \text{top} & \text{ag'ir'iz} \end{cases}$

Recurrence Relation

$$P[i, w] = \begin{cases} P[i, w] = 0 & \text{if } i=0 \text{ or } w=0 \\ P[i-1, w] & \text{if } w_i > w \text{ not taken} \\ \max(v_i + P[i-1, w-w_i], P[i-1, w]) & \text{al madim} \end{cases}$$

$\downarrow$ satir $\downarrow$ w. satir $\downarrow$ bazan,

Edit Distance

① grappe

graph, graft, grail, giraffe

② Computational Biology

ACGTGGA

AGTAGA

I N T E ~~E~~ N T I O N

~~E~~ E X E < U T I O N
↓ ↓ ↓ ↓ ↓
SI deşis d eklo deşis

toplam distance $\rightarrow 5$

① Substiteute $\rightarrow$ same
 $\rightarrow$ change $change(x, i, a) \rightarrow x_1 \dots x_{i-1} a x_{i+1} \dots x_n$

$x_i = y_j \rightarrow ED[i, j] = ED[i, j]$

$x_i \neq y_j \rightarrow ED[i, j] = ED[i-1, j-1]$

② Delete: $x_1, x_2 \dots x_{i-1}, x_{i+1} \dots x_n$

$x_i \neq y_j \rightarrow ED[i, j] = ED[i-1, j] + 1$

③ Insert: $x_1, x_2 \dots x_i, a, x_{i+1} \dots x_n$

$ED[i, j] = ED[i, j-1] + 1$

A E L M A
E
L
M
A

hops: hepsizle

kasliş,

$ED[i, j]$
x y

Rekürans Bağntısı

$ED[i, j] = \min(ED[i, j-1] + 1, ED[i-1, j] + 1, ED[i-1, j-1] + 1)$

$\rightarrow$ eğer farklıysa

$\rightarrow \min \begin{cases} ED[i-1][j] + 1 & \text{or} \quad \uparrow \text{ deletion} \end{cases}$

$\begin{cases} ED[i][j-1] + 1 & \text{or} \quad \leftarrow \text{ insertion} \end{cases}$

$\begin{cases} ED[i-1][j-1] + \text{cost} \rightarrow x_i = y_j \quad \text{cost} = 0 \\ \text{else} \quad \text{cost} = 1 \end{cases}$

$\rightarrow$ Substitution

$$y = MATHS$$
$$O(N^2) \approx O(N^2)$$

ARTS

festen

MATHS

gidel. l. i. i. i.

Subsequence of a word

common subsequence

yabamoz

502 942

502

ya z

Brute force $\rightarrow O(n! 2^n)$

$$\begin{array}{l}
 X: \text{ATCTGAT} \\
 Y: \text{TGCATA}
 \end{array}
 \left. \vphantom{\begin{array}{l} X: \text{ATCTGAT} \\ Y: \text{TGCATA} \end{array}} \right\} LCS(X, Y) = \begin{array}{c} \text{TCTA} \\ \text{or} \\ \text{TGAT} \end{array}$$

$L \subseteq [i, j]$ $x[i]$ vs $y[j]$ se
kader longest common subsequence

~~a~~ $x: ACGT \quad y: GC \rightarrow LCST[i,j] = 1$

Case 2: $x_i \neq y_j$

- $\rightarrow x_i$ is not part of $CS \quad LCS[i-1, j]$
- $\rightarrow y_j$ " " " " " " $LCS[i, j-1]$

$$LCS[i, j] = \begin{cases} 0 & \text{if } i=0, j=0 \\ LCS[i-1, j-1] + 1 & \text{if } i>0, j>0 \text{ and } x_i = y_j \end{cases}$$
$$\max(LCS[i-1, i], LCS[i, j-1]) \quad i \neq j \Rightarrow x_i \neq y_j$$

$x = \text{"BACDB"}$ $y = \text{"BDCB"}$

	B	D	C	E
B	0	0	0	0
D	0	1	1	1
C	0	1	1	1
E	0	1	2	2

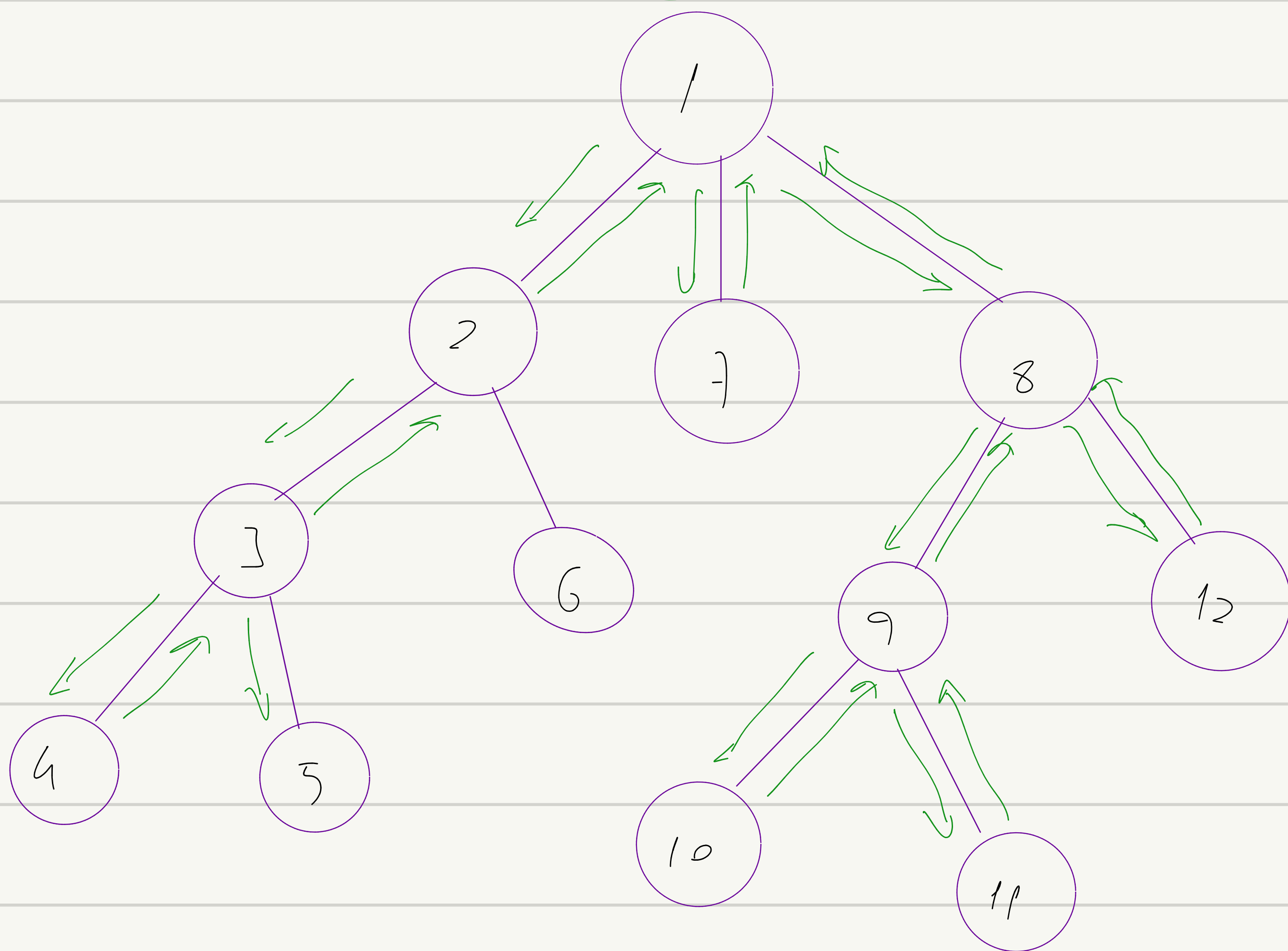
diagoneller:
algorithm
 $B \subset \mathbb{R}^n$

11. HAFTA

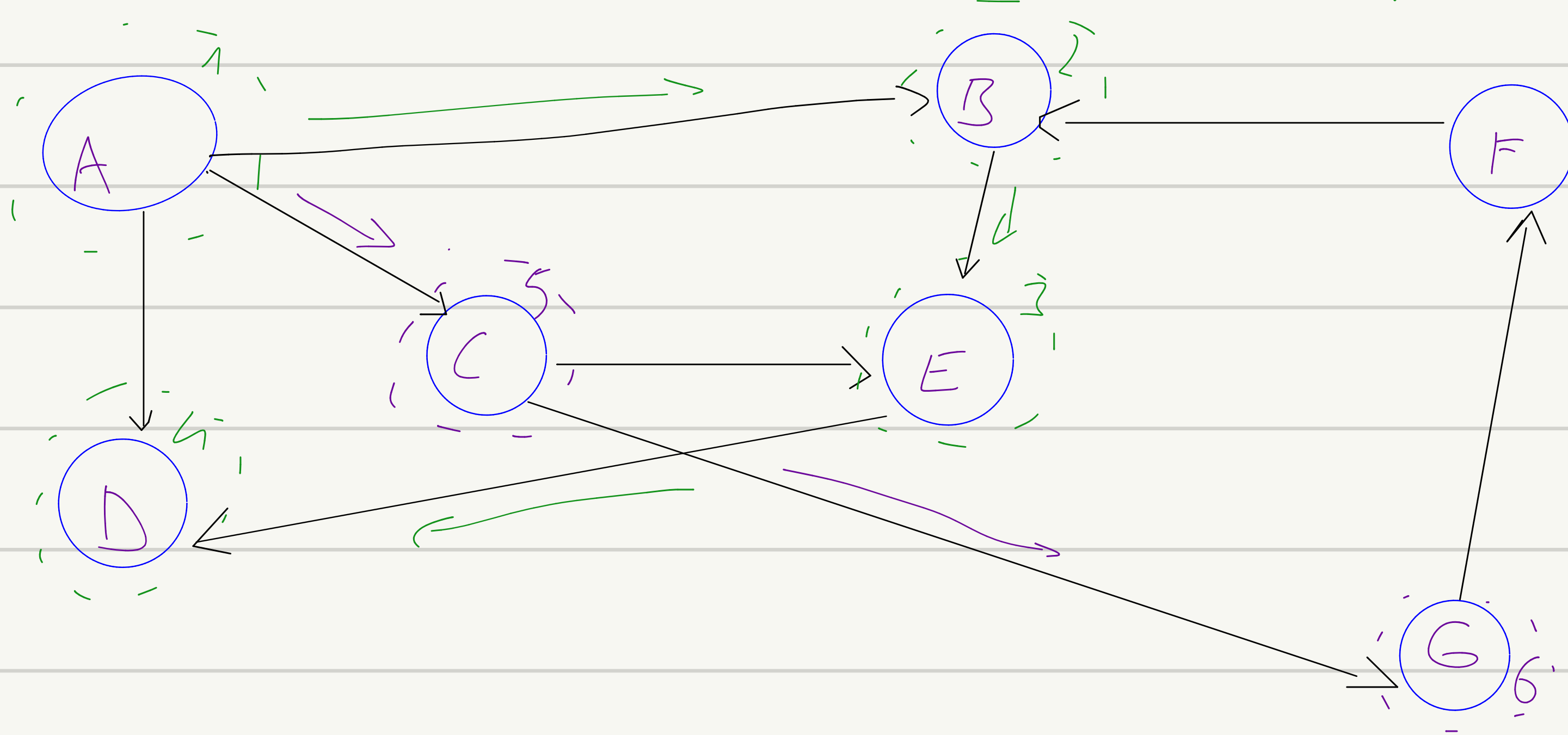
Graph traversal

→ komşuluk matrisi: → BFS → Queue
→ komşuluk listesi: → DFS

DFS → STACK



A'dan G'ye yol var mı?



A	B	C	D	E	F	G
Ø	Ø	Ø	Ø	Ø	0	Ø
1	1	1	1	1		1

visited

Recursive DFS
DFS(^{Graph}G, ^{initial point}s) {

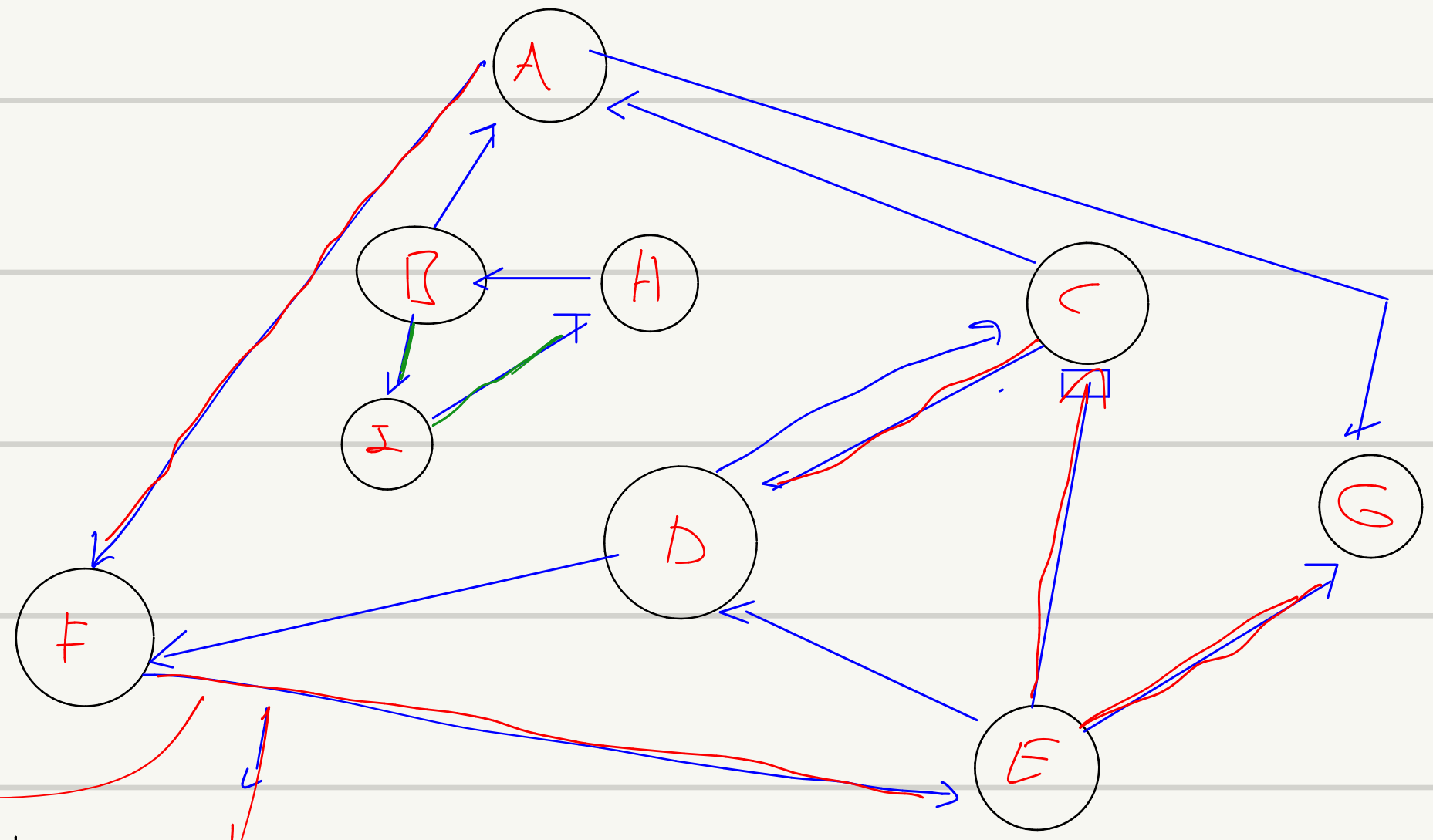
mark s as visited
for all neighbours w of s in graph G
if w is not visited
DFS(G, w) }

Iterative DFS

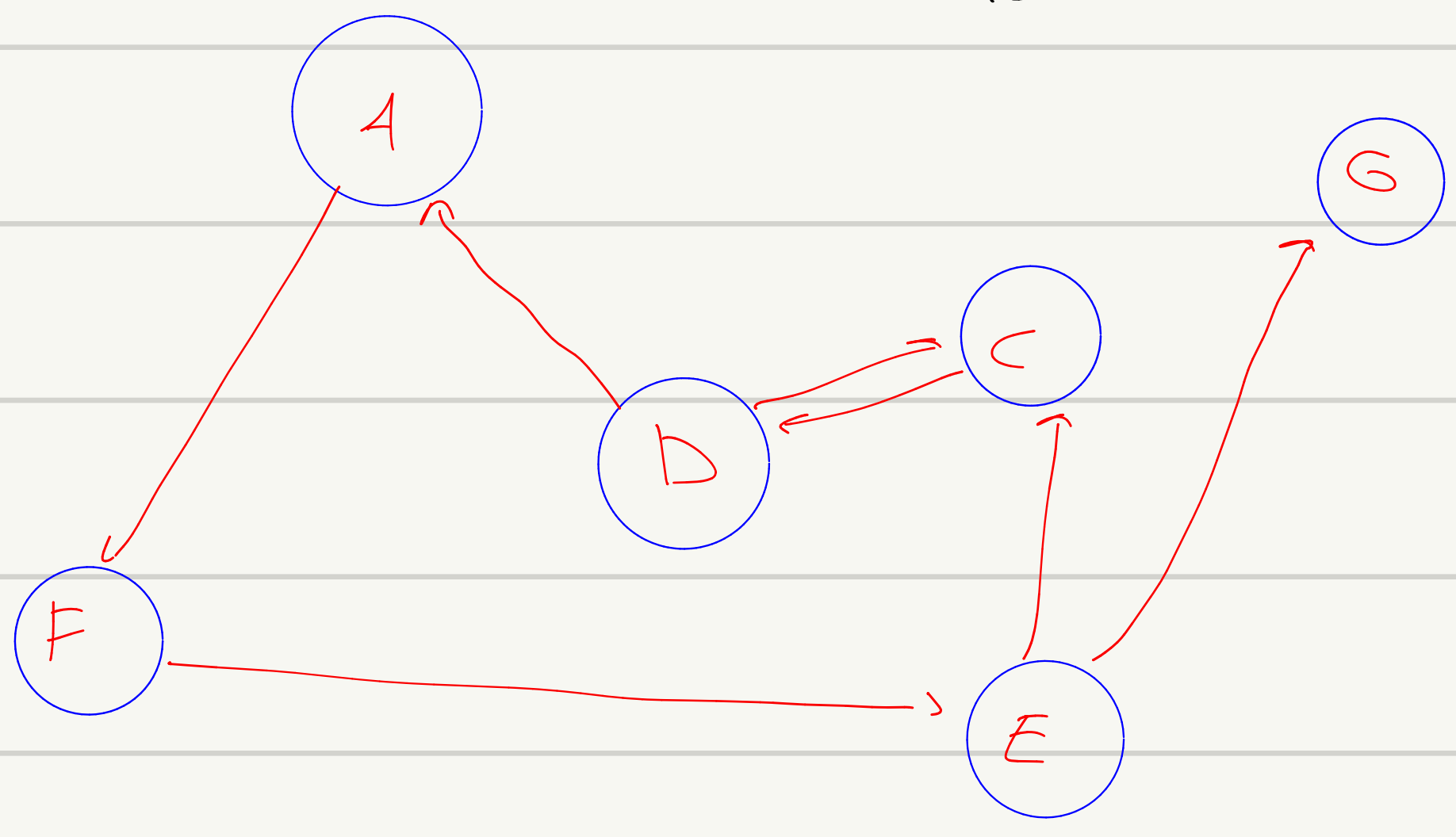
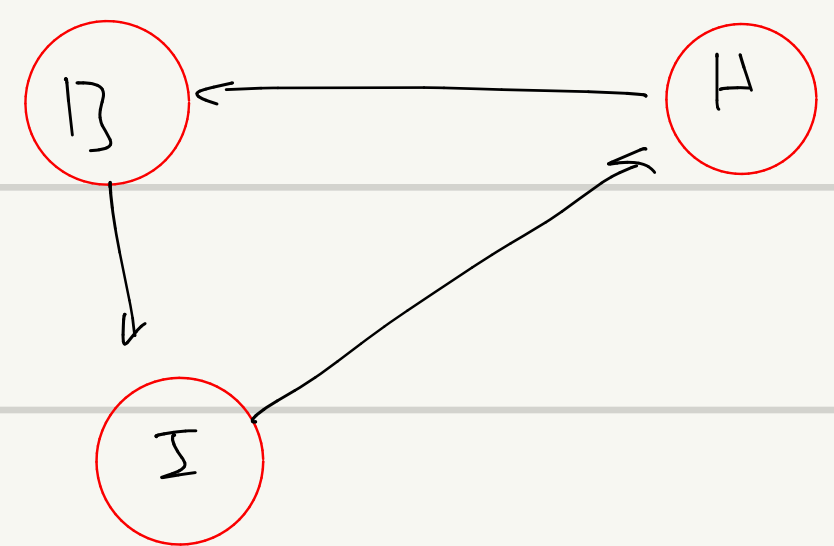
DFS(G, s)
push(s)
mark s as visited
while stack is not empty
v = pop()
for all neighbours w of v in graph G
if w is not visited
push w
mark w as visited

Connected Component

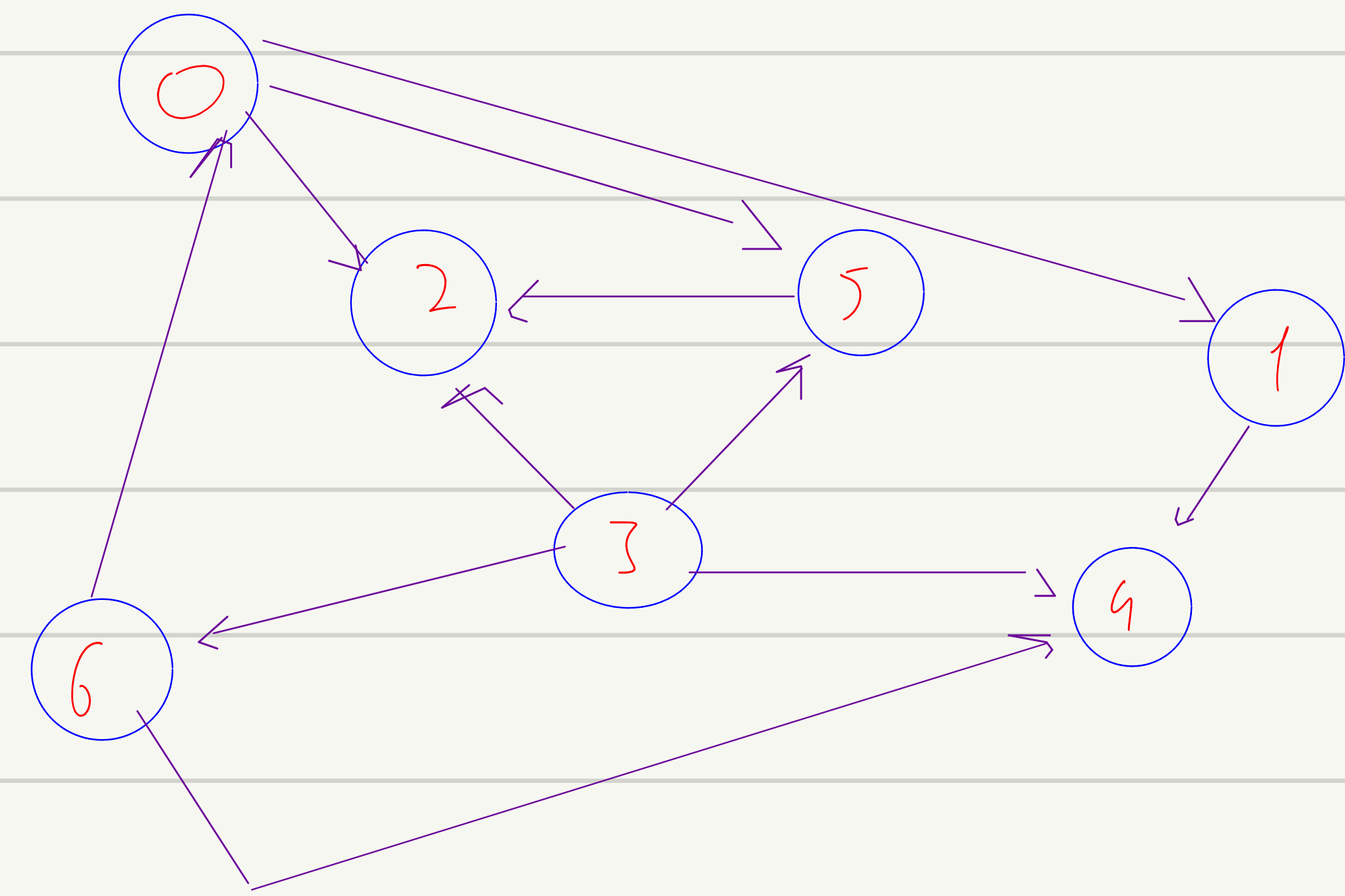
Yönlü Graph



Not A dan erişilebilir normalde 1 component



Topological Sort DFS $\rightarrow$ Acyclic $\vee$ Directed graph



Adjacency list

$0 \rightarrow 1 \rightarrow 2 \rightarrow 5$

$1 \rightarrow 4$

$2 \rightarrow x$

$3 \rightarrow 2 \rightarrow 4 \rightarrow 5 \rightarrow 6$

$4 \rightarrow x$

$5 \rightarrow 2$

$6 \rightarrow 0 \rightarrow 4$

indegree $\rightarrow 0$ in 1 (6 done)

outdegree $\rightarrow 0$ " 3 (1, 2, 5)

dfs(0)

dfs(1)

dfs(4)

4 done

1 done

dfs(2)

2 done

dfs(5)

check 2

5 done

0 done

dfs(3)

c 2

c 4

c 5

dfs(6)

c 0

c 4

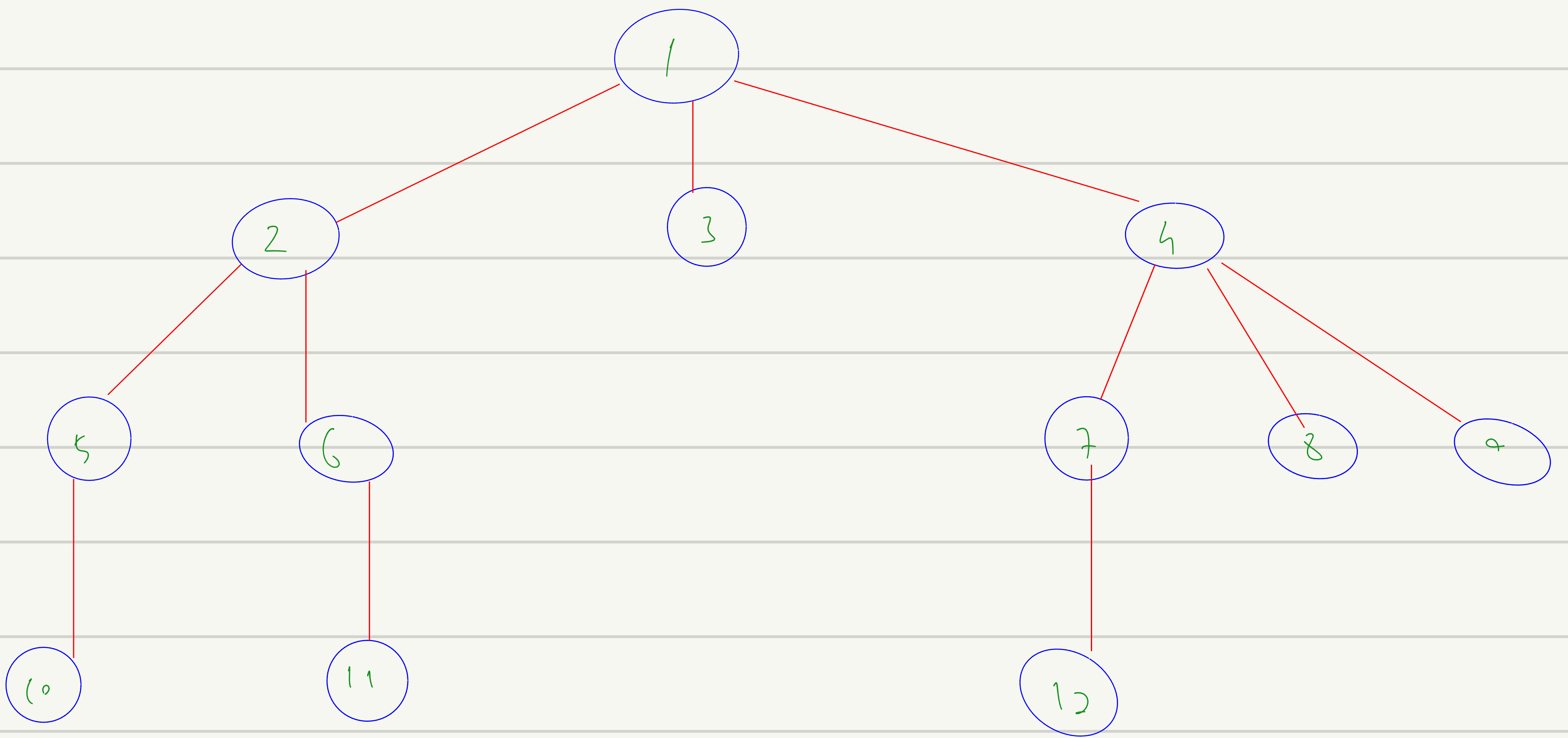
6 done

3 done



flood fill, cycle detection, topological sort, graph problems DFS
is graph, connected component de

BFS $\rightarrow$ Queue



BFS(G, s) {

enqueue(s)

mark as s visited

while (Queue is not empty)

$v = \text{dequeue}(s)$

for all neighbors w of v in G

if w isn't visited

enqueue w

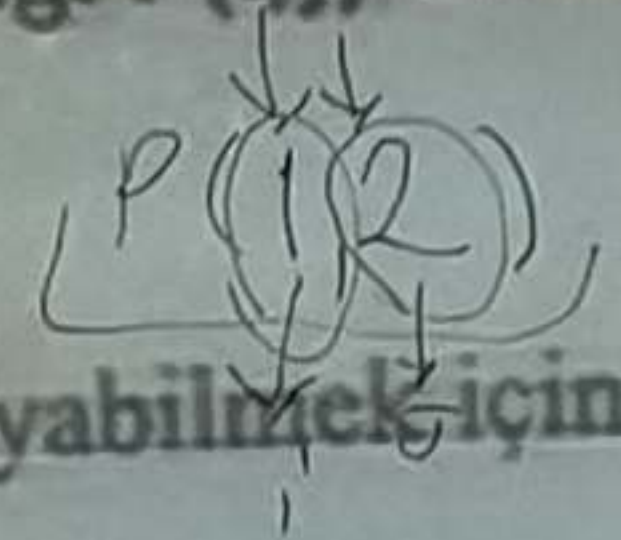
mark as w visited

}

Dinamik Programlama Örnek:

L A B

A ve B takımları, içlerinden biri n galibiyet alana kadar maç yapacaklardır. A takımının bir maçı kazanma olasılığı her maç için p , kaybetme olasılığı ise $1-p$ 'dir. Dolayısıyla beraberlik ihtimali mevcut değildir. A takımının seriyi kazanmak için i tane daha maç kazanması, B takımının da seriyi kazanmak için j tane maç kazanması gereken durumda A'nin seriyi kazanma olasılığı $P(i,j)$ 'dir.



- $P(i,j)$ 'yi kullanarak dinamik programlama ile algoritma tasarlayabilmek için rekürans bağıntısını yazınız.

- A takımının bir maçı kazanma olasılığının 0.4 olduğu durumda 7 maçlık seride (4 alan kazanır) A takımının kazanma ihtimalini hesaplayınız. (Dinamik programlama yaklaşımını kullanınız.)

$$P(i,j) = p * P(i-1,j) + (1-p) * P(i,j-1) \quad i,j > 0$$

$$P(0,j) = 1 \quad i = 0$$

$$P(i,0) = 0 \quad j = 0$$

		j				
		0	1	2	3	4
i	0	0	↓ 1	↓ 1	↓ 1	1
	1	0	→ 0.4 * (1-p)	→ 0.64 * p	→ 0.784	0.8704
	2	0	0.16	0.352	0.5248	0.66304
	3	0	0.064	0.1792	0.31744	0.45568
	4	0	0.0256	0.08704	0.1792	0.287776

P(p)

for (i=0; i<n; i++)

$$diz[i][0] = 0;$$

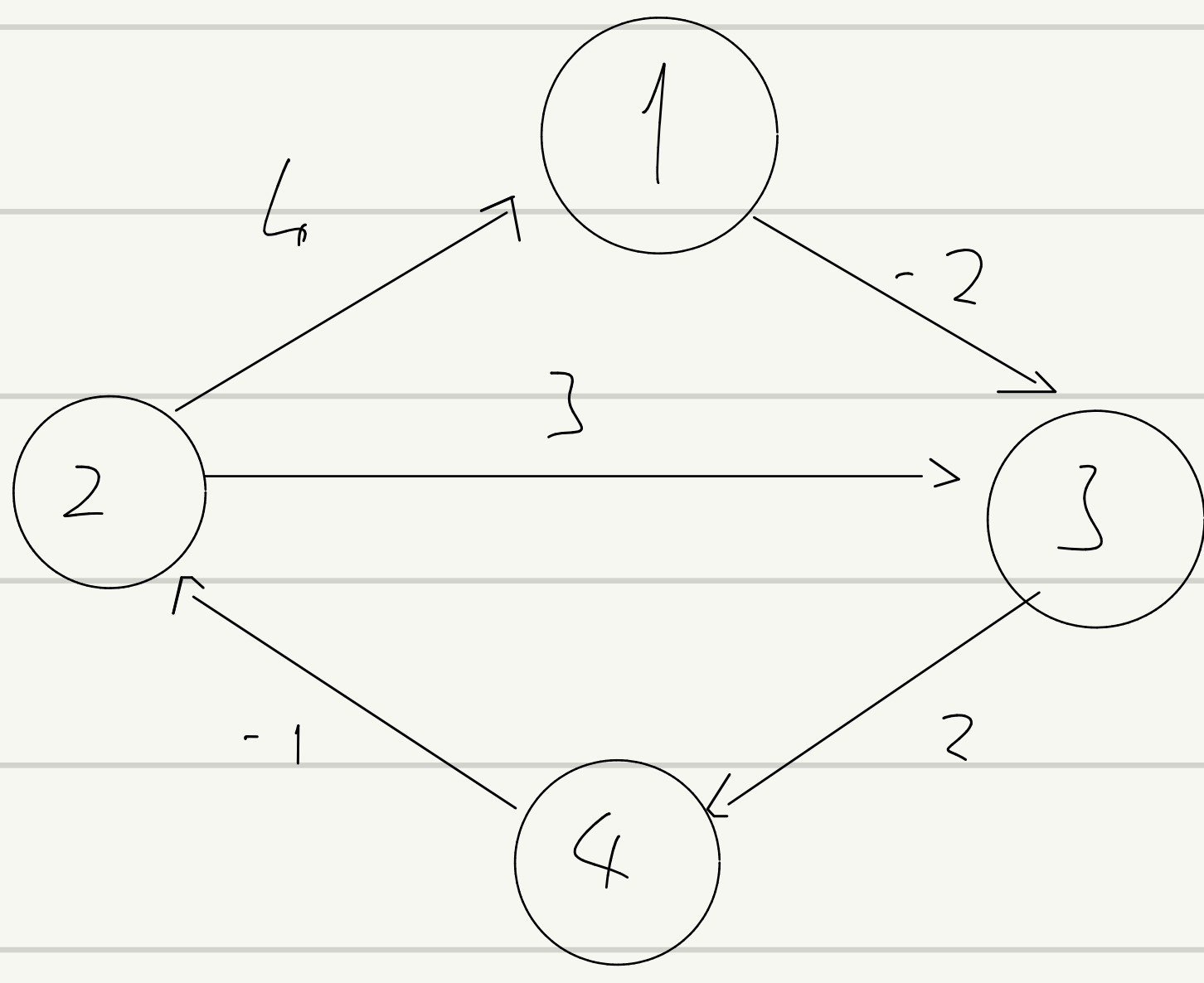
$$diz[0][i] = 1;$$

for (i=1; i<n; i++)

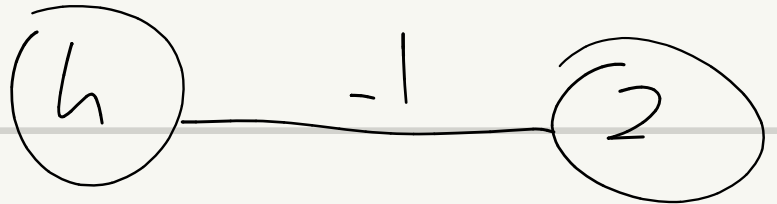
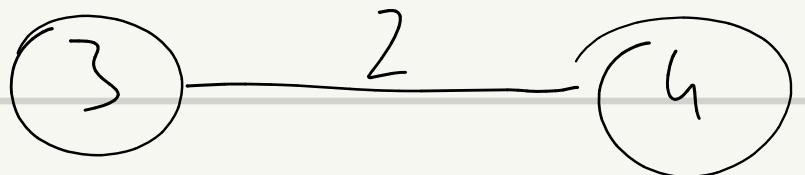
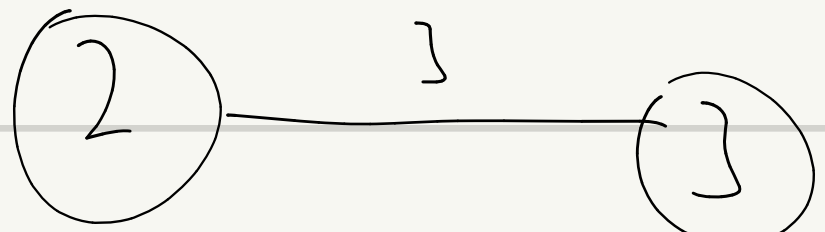
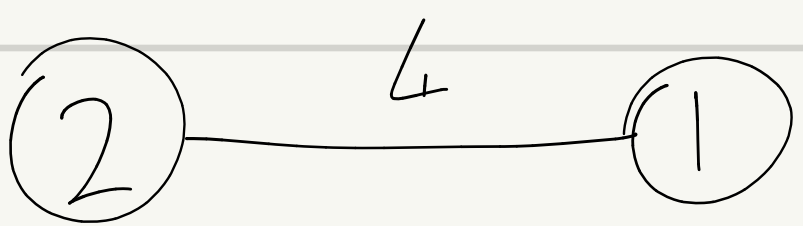
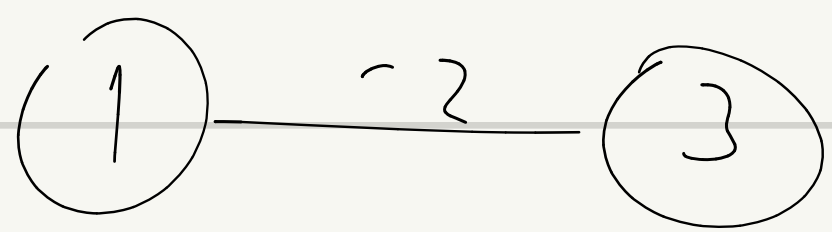
for (j=0; j<n; j++)

$$diz[i][j] = diz[i-1][j] * p + diz[i][j-1] * (1-p)$$

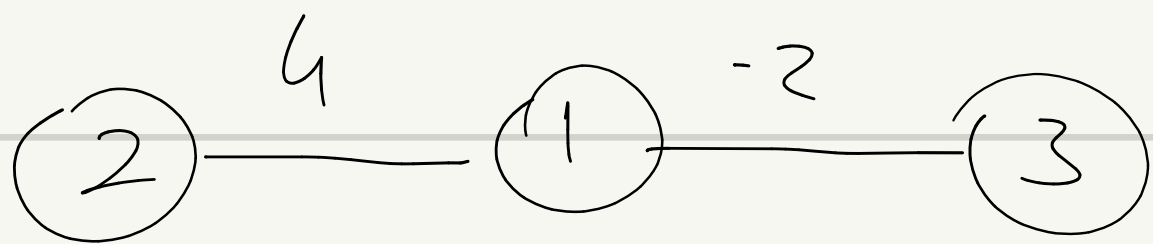
Graph - Floyd Warshall



k = 0



k = 1



$$\min(\text{distance}[i][j], \text{dist}(i, k) + \text{dist}(k, j))$$

↓

$$\text{dist}[2, 3] = 3$$
$$\text{dist}[2, 1] + \text{dist}[1, 3]$$

↓

k=0	1	2	3	4
1	0	∞	-2	∞
2	4	0	3	∞
3	∞	∞	0	2
4	∞	-1	∞	0

k=1	1	2	3	4
1	0	∞	-2	∞
2	4	0	2	∞
3	∞	∞	0	2
4	∞	-1	∞	0

k=2	1	2	3	4
1	0	∞	-2	∞
2	4	0	2	∞
3	∞	∞	0	2
4	3	-1	1	0

$W(\text{graph}, V, E)$

```
for i = 0; i < v; i++
  for j = 0; j < v; j++
    matrix[i][j] =  $\infty$ 
```

```
for each edge (u, v)
  dist[u][v] = u2v_k_k[u][v]
```

```
for i = 0; i < v; i++
  dist[v][v] = 0
```

```
for k = 0; k < v; k++
  for j = 0; j < v; j++
    for i = 0; i < v; i++
      if (dist[i][j] > dist[i][k] + dist[k][j])
        dist[i][j] = dist[i][k] + dist[k][j]
```

k_g

4: 20-2

$5k_g$

$\emptyset$ 1

2

2 2 1

$\nearrow$ 2

3

3 5 1

$\nearrow$ 3

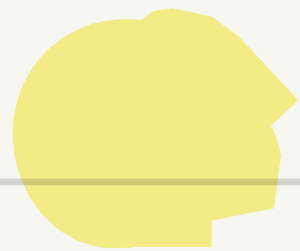
1

1 3 1

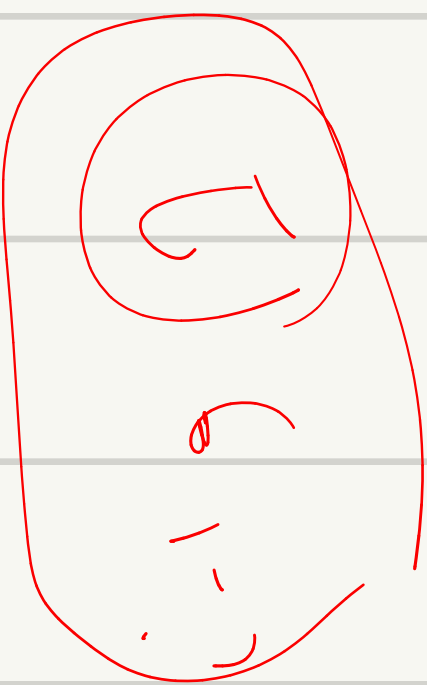
$\nearrow$ 4

1

1 2 1



k_g



icm

	0	1	2	3	4	5
0	0	0	0	0	0	0
1	0	0	2 0	2 0	2 0	2 0
2	0	0	2 0	3 0	4 0	5 0
3	0	1 0	2 0	3 0	4 0	5 0
4	0	1 0	2 0	3 0	4 0	5 0

1 2 3 4 5

0 1

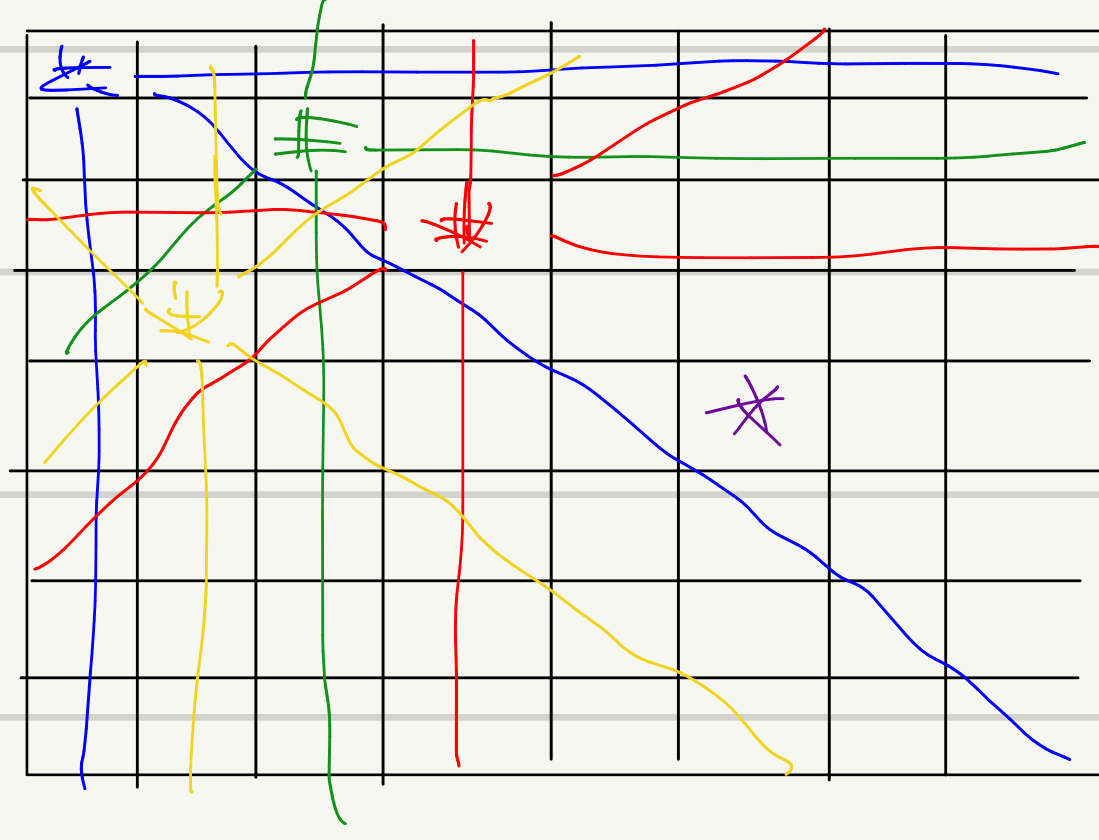
2 3

4. ✓
3. ✓
2. ✓

12. Hafta

Back tracking

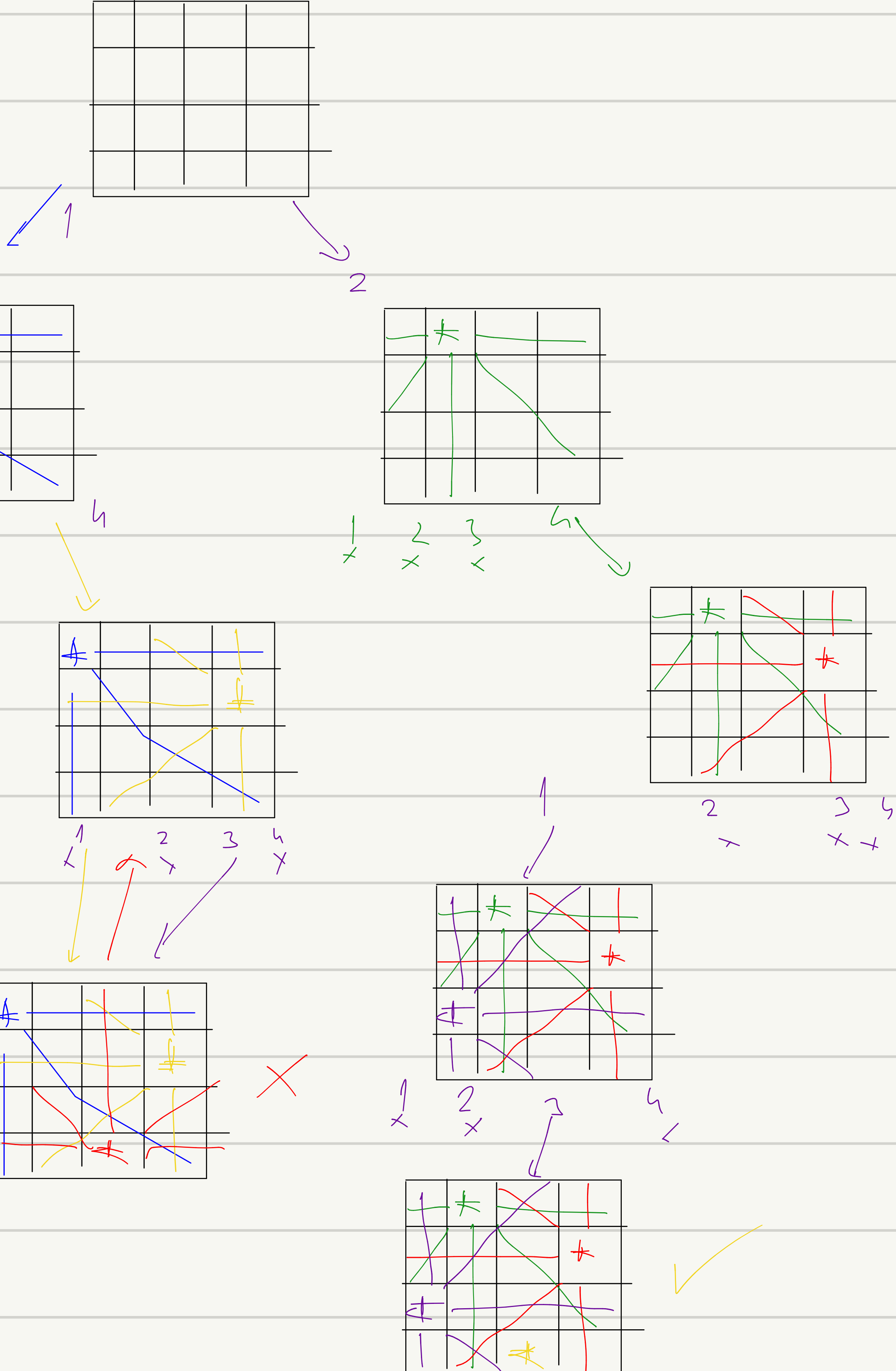
N-Queen Algorithm



Her merte 8 eune vezini yerlestiren, yerlesmesince geri donmek gerekiyor.

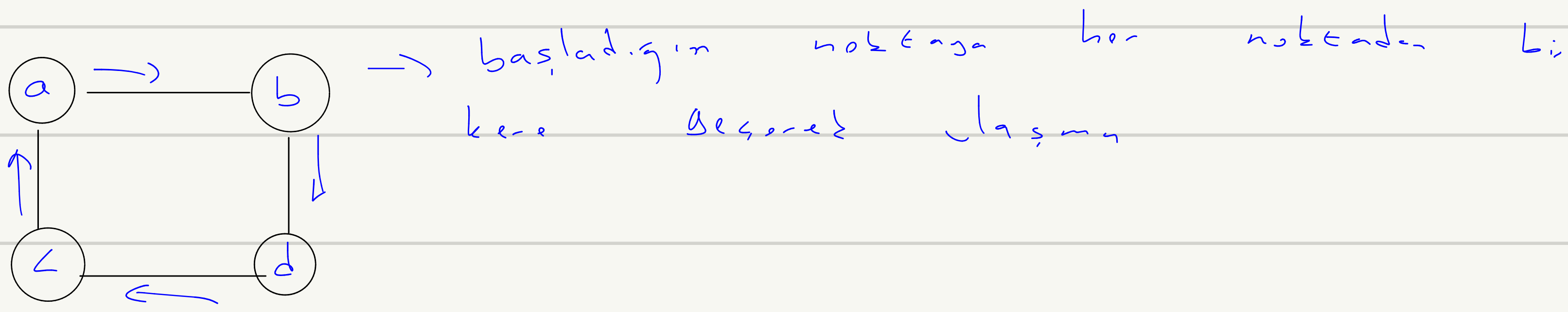
- ① Butun parca (64) $\rightarrow$ 4 milyon deneme
- ② Sadece kontrol yaparsak $n \rightarrow 8^8 \rightarrow 16$ milyon
- ③ Sadece ve sadece kontrol yaparsak $\rightarrow n!$ $\rightarrow$ 40 bin
- ④ Yerlestiremediginde bir oncedeki duruma donme islemi $\rightarrow$ 2 bin
 $\hookrightarrow$ back tracking

State space tree

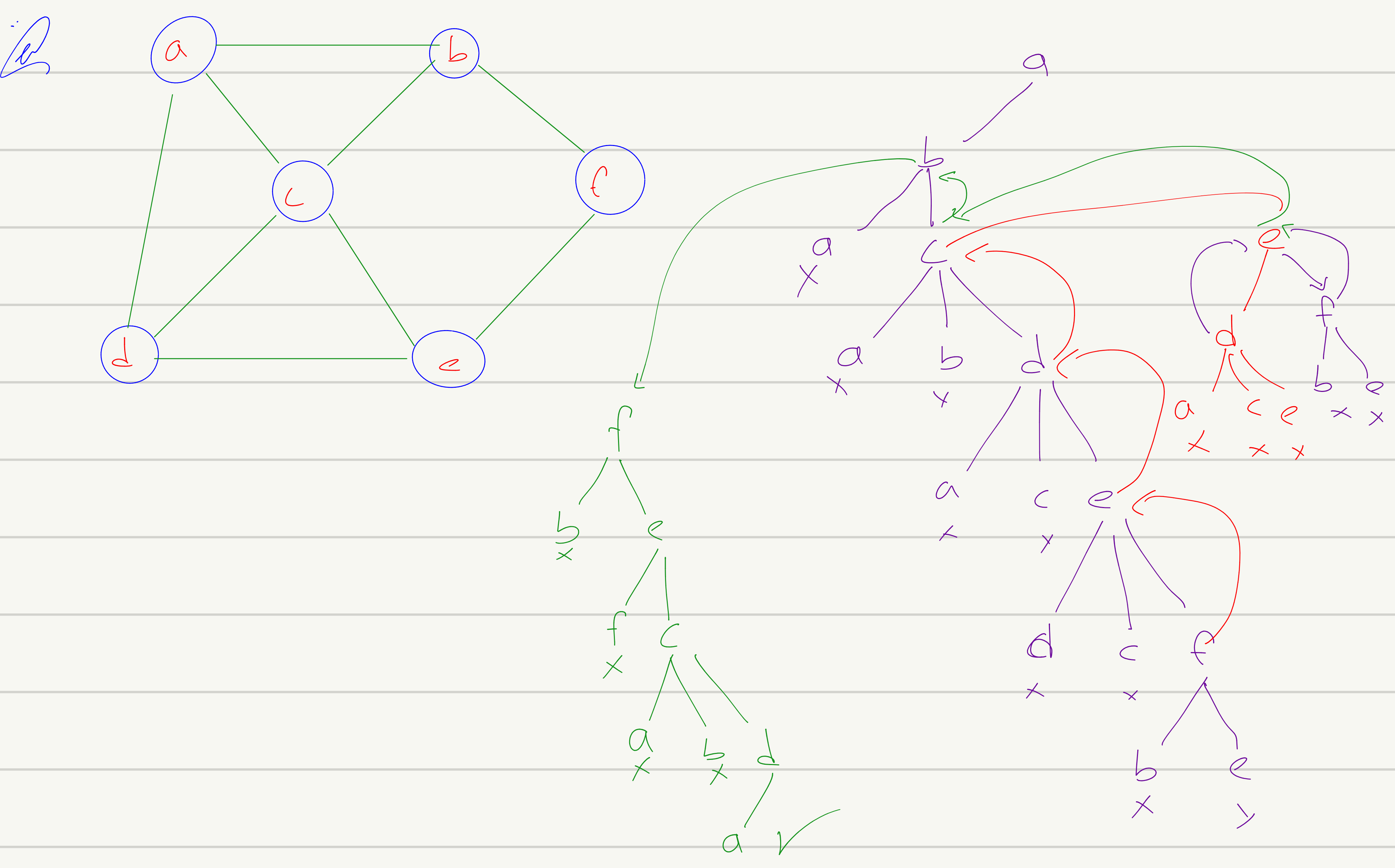


Sütun
deneyim

Hamiltonian Circuit $\rightarrow$ DFS Gib;



State space tree

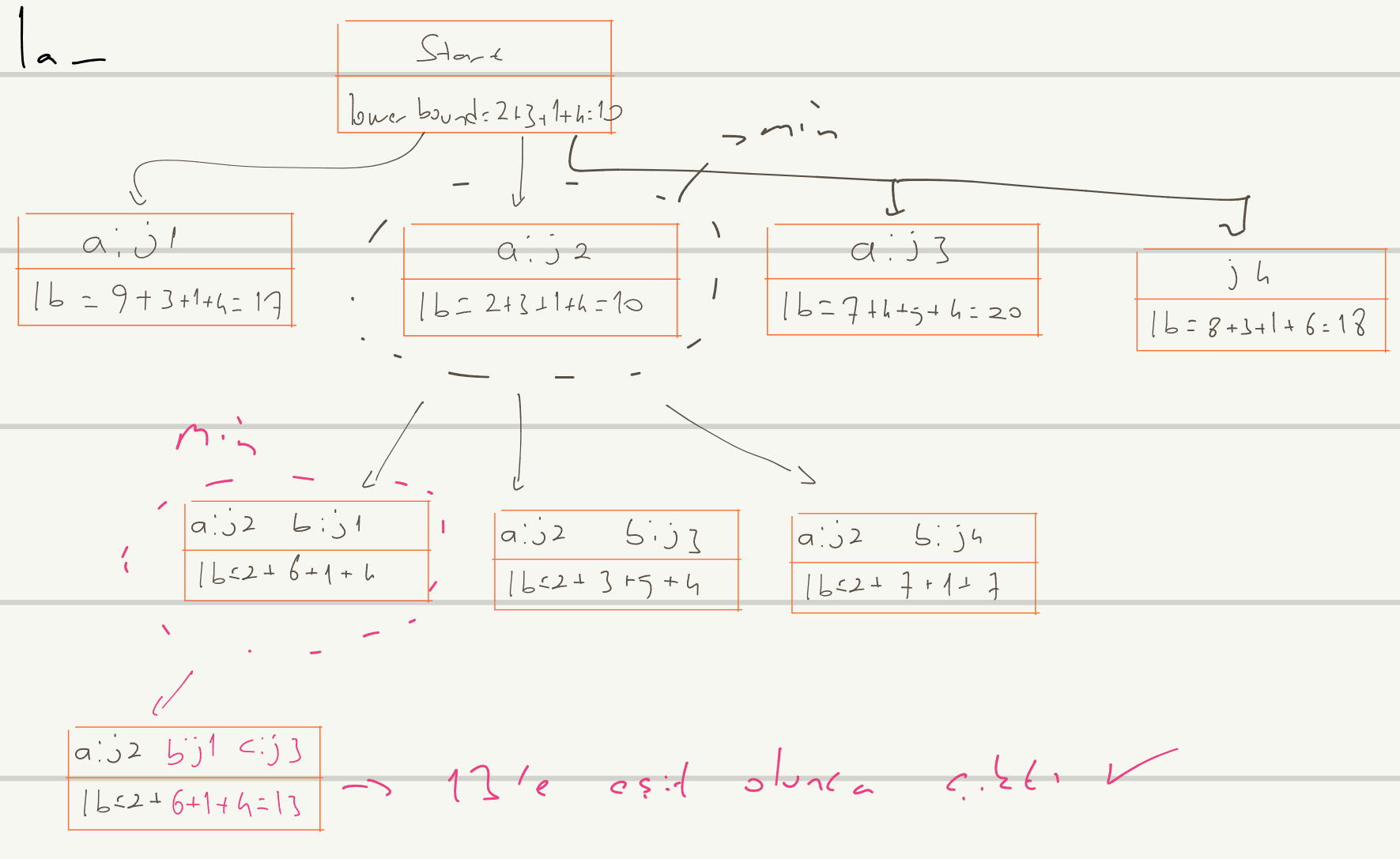


Branch and Bound $\rightarrow$ BFS

State Space Tree

h kişi var ve h işe yerleştirilebilir

	Job 1	Job 2	Job 3	Job 4	
$C =$	9	2	7	8	ahmet mehmet cafer serket
$\downarrow$	6	4	3	7	
$\downarrow$	5	8	1	8	
Perm	7	6	9	4	



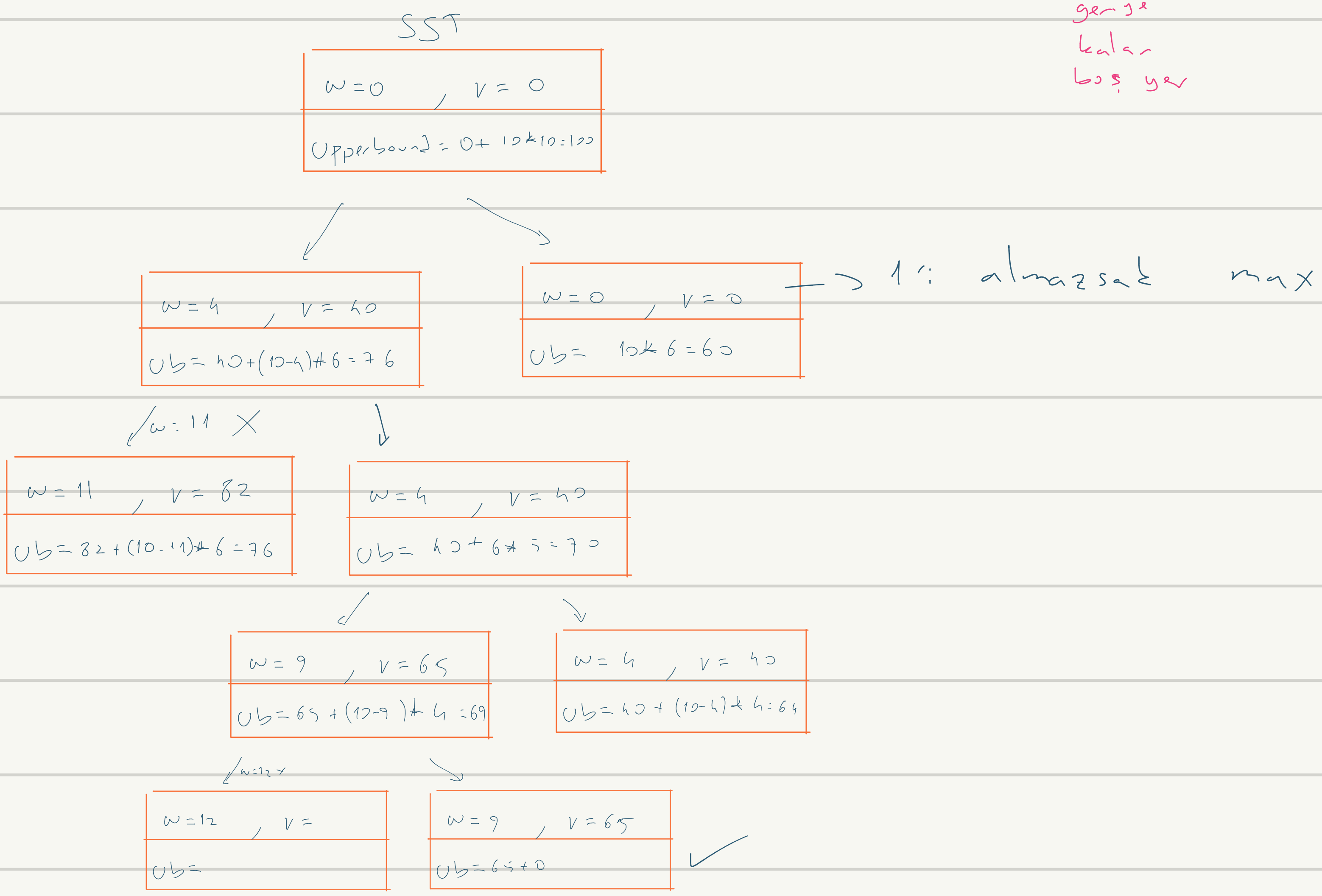
Knapsack Problem

W = 10 kg

item	weight	value	v_i/w_i
1	4	40	10
2	7	42	6
3	3	25	5
4	3	12	4

$v_1/w_1 > v_2/w_2 > v_3/w_3 > \dots$:s. in upper bound = $v + (W-w)(v_{i+1}/w_{i+1})$

anlık kazanç,
geriye kalan boş yer
den gelecek olan



13. HAFTA SON

Polynomial $\rightarrow O(n^{\text{miljon}}) \rightarrow$ efektif değil

efektifler $\rightarrow O(n^2), O(n^3), O(n \log n)$

efektif olmayanlar $\rightarrow O(2^n), O(n^n), O(n!), O(n^{\text{miljon}})$

$n^{\text{miljon}} \rightarrow$ trackable
 $n^{\log n} \rightarrow$ in " " " " " trackable

NP $\rightarrow$ Non det. polynomial

Traveling Salesman $\rightarrow$ Başladığın yerden her gere'len fazla uğrayarak, her yeri gezmek.

Hamiltonian circuit $\rightarrow$ Kenar ağırlı. Traveling Salesman

her durum denenmelidir

$\rightarrow$ Dynamic $\rightarrow 2^n \cdot n$
 $\rightarrow$ Brute force $\rightarrow n \cdot n!$

Problems

Decision Problems

Optimization Problems

Deterministic $\rightarrow$ çıktı belli

$\rightarrow$ max element

```
NSort(A,B&
for(i=0;i<N;i++&
B[i]=0;

for(i=0;i<N;i++&
j=rand(1..N&
if(B[j]!=0& then failure
B[j]=A[i]
end for
for(i=0;i<n-1;i++&
if(B[i]>B[i+1]& then failure;
end for
print(B&
success
O(1&
```

Nondeterministic $\rightarrow$ çıktı belirsiz

$\rightarrow$ en yakın gel

```
NSearch(A,x&
j=choice(1..n&  $\rightarrow$  ① guessing
if(A[j]=x&
print(j&
success  $\rightarrow$  ② Verification
endif
print(0&
failure  $\rightarrow$  ③ Output
```

Complexity $O(1&$

$\uparrow$
NP

$P \subseteq NP$

$NP \rightarrow$ polinomsal zamanda doğrulanabilir algoritmalar